

ENHANCING SDN NETWORK SECURITY USING GAME THEORY

Prepared by

Nagham Attieh

Bayan Alnassari

Supervised by

Dr. Wassim Juneidi

Eng. Mohammad Yamen Hallak

Table of Contents

1	Chapter One: Introduction	10
1.1	Background	11
1.2	Problem Statement	12
1.3	Project Motivation	12
1.4	Aim of the Project	13
1.5	Project Objectives	14
1.6	Literature Basis	15
1.7	Research Gap	17
1.8	Challenges	18
1.9	Practical Applications	19
1.10	Chapter Summary	19
2	Chapter Two: Theoretical Framework	20
2.1	Software-Defined Networking and Security Architecture	21
2.2	OpenFlow-Based Monitoring and Enforcement	22
2.3	Intrusion Detection and SDN Security Integration	22
2.4	Game Theory and Attacker-Defender Interaction	23
2.5	Multi-Stage Cyberattacks	24
2.6	Markov-Based Attack Stage Modeling	26
2.7	Artificial Intelligence for Network Intrusion Detection	27
2.8	Network-Traffic Feature Engineering	27
2.9	Principal Component Analysis	29
2.10	Attack Pattern Segmentation	30
2.11	Ensemble and Stacking Classification	31
2.12	Hierarchical Attack Classification	33
2.13	Prediction Confidence and Joint Scoring	33

2.14	Integration of AI Detection with SDN Defense	34
2.15	Chapter Summary	36
3	Chapter Three: Working Environment	37
3.1	Introduction	38
3.2	Overall Project Implementation Environment.....	38
3.3	Software and Development Components	40
3.4	Original SDN Experimental Environment.....	42
3.5	Virtual Machines and Network Interface Layout.....	43
3.6	SDN Topology and Security Components	45
3.7	Artificial Intelligence Development Environment.....	47
3.8	Dataset Selection and Target Attack Families.....	48
3.9	AI Data-Processing Pipeline	50
3.10	Data Cleaning and Label Preparation.....	51
3.11	Port-Variance Feature Engineering.....	53
3.12	Data Standardization and PCA Preparation	54
3.13	Attack Pattern Segmentation.....	56
3.14	Stacking Ensemble Implementation	59
3.15	Hierarchical CatBoost Implementation.....	59
3.16	Prediction Output and Confidence Generation	60
3.17	AI-to-Game-Theory Integration	61
3.18	Dynamic Game Theory Decision Module.....	62
3.19	SDN-Based Response Generation	63
3.20	Chapter Summary	63
4	Chapter Four: Proposed Solution	64
4.1	Introduction	65
4.2	Overall Integrated Solution Architecture.....	65
4.3	Multi-Stage Attack Tracking and Representation	66
4.3.1	Operational Security States	67

4.3.2	Stage Classification and Multi-Stage Transition Detection	68
4.3.3	Markov-Based Temporal Modeling	69
4.3.4	Multi-Stage Evidence in the Defense Process.....	71
4.4	AI-Based Attack Detection and Threat Representation	72
4.4.1	Attack Pattern Segmentation.....	72
4.4.2	Hierarchical Attack Classification	73
4.4.3	Confidence-Aware Hierarchical Prediction	74
4.4.4	AI-to-Defense Acceptance Condition.....	75
4.5	Integrated Multi-Stage and AI Decision Evidence.....	77
4.6	Dynamic Game-Theoretic Defense Decision.....	78
4.6.1	Evidence-Based Attacker Beliefs	78
4.6.2	Dynamic Malicious Reputation	79
4.6.3	Defender Payoff Model.....	80
4.6.4	Expected Utility Calculation	81
4.6.5	Optimal Defense Selection.....	82
4.7	Adaptive Mitigation Strategy	82
4.8	Chapter Summary	83
5	Chapter Five: Experimental Results and Evaluation	84
5.1	Introduction	85
5.2	Multi-Stage Attack Evaluation Results	85
5.3	AI-Based Hierarchical Classification Results.....	88
5.4	Weak Subpattern Analysis Results.....	90
5.5	AI-to-VMware and Game Theory Integration Results	91
5.6	Behavioral Interpretation of Generated Subpatterns	94
5.7	Game Theory Decision Results.....	97
5.8	SDN Action Mapping Results.....	100
5.9	OpenFlow Rule Generation Results	101
5.10	Comparison with Graduation 1.....	102

5.11 Chapter Summary	105
6 Chapter Six: Conclusion.....	106
References.....	107

List of Tables

Table 1.1 Summary of the adopted studies supporting the project.....	16
Table 2.1 Game-Theoretic Mapping Used in the Proposed Project.	24
Table 3.1 Software and Development Components Used in the Complete Project	40
Table 3.2 Virtual-Machine Interfaces Used for Management and Traffic Monitoring	44
Table 3.3 Traffic Families Selected for the Artificial Intelligence Pipeline	49
Table 3.4 . Standardization and PCA Configuration	55
Table 3.5 Generated Behavioral Subpatterns.....	57
Table 4.1 Operational Security States Used in the Multi-Stage Tracking Mechanism	67
Table 4.2 Markov Transition Probability Matrix Used in the Multi-Stage Model.....	70
Table 4.3 Hierarchical Classification Levels.....	74
Table 4.4 Confidence Thresholds Used in the AI Decision Pipeline	76
Table 4.5 presents the defender payoff matrix implemented in the current Game Theory engine....	81
Table 4.6 Adaptive Mitigation Actions.....	83
Table 5.1 summarizes the main differences between the two project.....	103

List of Figure

Figure 2.1 Basic SDN architecture used as the theoretical basis of the project, adapted from [1]....	22
Figure 2.2 Illustrative relationship between multi-stage attack progression and Markov-based security-state tracking in the project.....	25
Figure 2.3 Port-Variance Distributions for Benign and Multi-Stage Attack Patterns Using a Sliding Window of Size 1,000, Adapted from [8].....	28
Figure 2.4 Overview of the Attack Pattern Segmentation-Based Network Attack Detection Technique, Adapted from [8].....	31
Figure 2.5 Two-Level Stacking Ensemble Used for Segmented Attack-Pattern Classification	32

Figure 2.6 End-to-End Integration of AI Detection, Game Theory, and SDN-Based Defense.....	35
Figure 3.1 Complete AI-Driven Multi-Stage Attack Detection and SDN Defense Environment	39
Figure 3.2 . Original SDN Monitoring, Detection, Decision, and Enforcement Environment	43
Figure 3.3 Network Interfaces Configured on the Mininet Virtual Machine	44
Figure 3.4 Management and Monitoring Interfaces Configured on the Snort Virtual Machine	44
Figure 3.5 SDN Topology and Security-Component Integration.....	46
Figure 3.6 Modular Organization and Execution Output of the Artificial Intelligence Project.....	48
Figure 3.7 Target Traffic-Family Selection and Dataset Generation Output	50
Figure 3.8 Final Data-Cleaning Summary and Cleaned-Dataset Generation Output	52
Figure 3.9 Port-Variance Feature Engineering Summary and Enhanced-Dataset Generation Output	54
Figure 3.10 Explained-Variance Summary and PCA Dataset Generation Output.....	56
Figure 3.11 Final Attack-Pattern Segmentation and Subpattern Distribution.....	58
Figure 4.1 Attack Pattern Segmentation Process in the Proposed AI Framework	73
Figure 5.1 Multi-Stage Attack Traffic Generation in Mininet	86
Figure 5.2 Snort Detection of Malicious Traffic during the Multi-Stage Scenario	86
Figure 5.3 . Multi-Stage Attack State Transition Detected by the Ryu Controller	87
Figure 5.4 Game-Theoretic Defense Decision Based on Multi-Stage Evidence.....	88
Figure 5.5 Validation and Test Performance of the Improved Hierarchical CatBoost Model	89
Figure 5.6 Low-Precision Subpattern Error Analysis.....	90
Figure 5.7 AI Detection Bridge Summary and Game Theory Input Generation	92
Figure 5.8 Successful Transmission of AI Detection Results to the VMware Environment.....	93
Figure 5.9 Successful Reception of AI Detection Results inside the Mininet VMware Environment ...	93
Figure 5.10 Reconstruction of Subpattern Centers in the Original Feature Space.....	95
Figure 5.11 Completed Original-Feature Interpretation of the Generated Subpatterns	96
Figure 5.12 Game Theory Decision Summary and Selected Defense Strategies.....	97
Figure 5.13 Example Game-Theoretic Decision Based on Attack Probabilities and Expected Utilities	98
Figure 5.14 SDN Action Mapping Summary.....	100
Figure 5.15 Example Final SDN Action Generated from the Game Theory Decision	101
Figure 5.16 Generated OpenFlow Meter and Flow-Rule Template.....	102

List of Terms and Abbreviations

Full Term	Abbreviation
Software-Defined Networking	SDN
Distributed Denial of Service	DDoS
Denial of Service	DoS
Intrusion Detection System	IDS
Virtual Machine	VM
Internet Control Message Protocol	ICMP
Internet Protocol	IP
Transmission Control Protocol	TCP
User Datagram Protocol	UDP
Round Trip Time	RTT
Packets Per Second	pps
Kilobits Per Second	kbps
Quality of Service	QoS
Central Processing Unit	CPU
Long Term Support	LTS
Rate Limiting Level 1	RL_1
Rate Limiting Level 2	RL_2
Rate Limiting Level 3	RL_3
Economic Denial of Sustainability	EDoS
Artificial Intelligence	AI
Principal Component Analysis	PCA
Principal Component	PC
Comma-Separated Values	CSV
JavaScript Object Notation	JSON
Hypertext Transfer Protocol	HTTP

Full Term	Abbreviation
Secure Shell	SSH
Support Vector Machine	SVM
F1-Score	F1

Abstract

Software-Defined Networking (SDN) enables centralized and programmable network control, but it also introduces security risks related to flooding, spoofing, Distributed Denial-of-Service (DDoS), and multi-stage attacks. This project presents an enhanced adaptive SDN security framework integrating Snort IDS, Artificial Intelligence, Game Theory, and OpenFlow-based mitigation.

The original phase established the SDN defense environment using Mininet, Open vSwitch, the Ryu Controller, Snort IDS, and an alert-forwarding mechanism. The current phase extends this architecture with multi-stage attack tracking and an AI-based detection pipeline using CIC-IDS-2018. After preprocessing and PCA, the selected traffic families are segmented into 22 behavioral subpatterns. A hierarchical CatBoost classifier identifies the traffic family and corresponding subpattern and generates confidence information that is transferred to the SDN environment. The Game Theory engine combines this AI evidence with network and multi-stage observations to select adaptive actions such as ALLOW, RL_1, RL_2, RL_3, or BLOCK, which are translated into OpenFlow-compatible mitigation actions.

Experimental results showed 98.05% family-level accuracy and 97.28% subpattern-level accuracy on the test set, with a macro F1-score of 93.44% and a weighted F1-score of 97.81%. The complete workflow successfully connected detailed AI-based attack identification with Game Theory decision making, SDN action mapping, and OpenFlow mitigation. Overall, the enhanced framework preserves the original SDN defense architecture while adding multi-stage awareness, behavioral attack classification, and confidence-aware AI integration.

Keywords: Software-Defined Networking, Artificial Intelligence, Multi-Stage Attacks, Game Theory, CatBoost, Snort IDS, OpenFlow, Adaptive Mitigation.

1 Chapter One: Introduction

1.1 Background

Software-Defined Networking (SDN) has emerged as a transformative networking paradigm that separates the control plane from the data plane and centralizes network intelligence within a programmable controller. This architectural shift has significantly improved network manageability, scalability, and flexibility, allowing administrators to configure policies, optimize traffic flows, and deploy services more efficiently than in traditional networks. As SDN continues to gain importance in cloud computing, virtualization, and security-aware infrastructures, its programmability has made it a promising platform for building more responsive and adaptive defense mechanisms [1], [2], [3].

Despite these advantages, SDN introduces new security concerns. The controller, as the logical core of the network, becomes a strategic target for adversaries. Among the most serious threats in this context are Distributed Denial-of-Service (DDoS) attacks, which attempt to overwhelm the control layer with malicious traffic, degrade network responsiveness, and disrupt service availability. The literature consistently emphasizes that the same centralization that gives SDN its strength can also become its greatest vulnerability if not properly protected [1], [3], [5].

In recent years, research has increasingly moved beyond static protection methods toward adaptive security frameworks that combine centralized control, intelligent traffic inspection, and dynamic response. This shift is particularly visible in studies that integrate SDN with Intrusion Detection Systems (IDSs), flow-based traffic monitoring, and game-theoretic decision models. These directions are highly relevant to the present project because they support the design of a framework that does not stop at attack detection, but extends toward strategic and context-aware mitigation [1], [4], [5].

Advanced cyberattacks may consist of multiple stages and behavioral patterns, some of which may resemble legitimate traffic or overlap statistically with other attacks. This makes it difficult for conventional Artificial Intelligence-based intrusion detection models to learn clear decision boundaries using only broad attack labels [8].

To address this limitation, Gong, Cho, and Choi [8] proposed attack pattern segmentation, in which clustering is used to divide broad attack behaviors into detailed subpatterns before classification. In parallel, Game Theory can model the changing interaction between an attacker and a defender across different attack stages [9]. Based on these two directions, the

present project extends the existing SDN security framework by integrating detailed multi-stage attack detection with the previously developed Game Theory and OpenFlow-based defense process.

1.2 Problem Statement

SDN provides centralized and programmable network control, but this centralization also exposes the controller and network resources to DDoS, flooding, and spoofing-related threats. Effective protection therefore requires not only detecting malicious traffic but also selecting an appropriate mitigation action without unnecessarily affecting legitimate communication [1], [3], [4].

A further challenge arises in multi-stage attack detection. Network flows belonging to the same attack family may contain several statistically distinct behaviors, while some attack stages may resemble benign traffic. Representing these behaviors using one broad class label can conceal important internal patterns and increase classification errors [8].

Therefore, the main problem addressed by this project is how to connect detailed multi-stage attack identification with an adaptive SDN defense process that uses the detected behavior as evidence for Game Theory-based decision making and OpenFlow mitigation.

1.3 Project Motivation

The motivation for this project is both practical and academic. From a practical perspective, modern programmable networks require more than passive monitoring. A useful security framework must be capable of observing malicious behavior, interpreting it correctly, and responding with a suitable mitigation action. This is particularly important in SDN, where the centralized controller gives the network the ability to enforce policies dynamically and at scale [1], [2].

From the original project perspective, the work was motivated by the need to connect intrusion detection with adaptive mitigation rather than treating them as separate processes.

This direction was supported by previous SDN studies that combine traffic inspection, controller-based reasoning, and game-theoretic defense selection [1], [4].

The original project phase established the SDN environment, Snort-based traffic inspection, the Game Theory decision engine, and OpenFlow-based mitigation. The current phase extends this framework from individual flooding and spoofing events toward more detailed analysis of multi-stage attack behavior.

The detection methodology proposed in [8] motivates the use of attack pattern segmentation to identify internal behavioral subpatterns rather than relying only on broad attack labels. Building on this idea, the project develops a hierarchical CatBoost mechanism that first predicts the attack family and then identifies its corresponding subpattern. The resulting detection information is subsequently transferred to the existing Game Theory-based defense process.

1.4 Aim of the Project

The main aim of this project is to enhance SDN network security by designing an adaptive DDoS-oriented defense framework that integrates the Ryu Controller, OpenFlow, Open vSwitch traffic mirroring, Snort IDS, and a game-theoretic decision engine. The proposed framework focuses on detecting and mitigating distributed ICMP flooding behavior and spoofing-related traffic in an emulated SDN environment. Instead of relying on a fixed threshold, the framework evaluates multiple indicators, including packet rate, Snort alert status, packet loss, RTT, spoofing indicators, and the number of suspicious sources, in order to select a proportional mitigation action.

The extended phase incorporates an Artificial Intelligence pipeline for multi-stage attack detection. Network-traffic data is preprocessed, transformed using PCA, and segmented into detailed behavioral subpatterns. The stacking-ensemble approach adopted from the reference detection methodology is evaluated, and an improved hierarchical CatBoost classifier is developed to first predict the attack family and then identify its corresponding subpattern. A joint confidence score combines family-level and subpattern-level prediction probabilities, and the resulting detection output is transferred through the detection–game bridge to support the existing stage-aware Game Theory defense process and subsequent OpenFlow mitigation.

1.5 Project Objectives

The project seeks to achieve the following objectives:

1. To build an emulated SDN environment using Mininet, Open vSwitch, and the Ryu Controller, while configuring traffic mirroring toward a dedicated Snort IDS environment.
2. To detect suspicious ICMP flooding, DDoS-oriented, and spoofing-related traffic using Snort and controller-observed network indicators, and to forward normalized Snort alerts to the Ryu-based defense environment.
3. To implement a Game Theory-based defense mechanism that evaluates the observed attack context and selects among ALLOW, RL_1, RL_2, RL_3, and BLOCK.
4. To enforce the selected defense strategy through OpenFlow-based rate limiting and drop rules.
5. To evaluate the original SDN defense framework using normal traffic, single-source flooding, multi-source DDoS-oriented scenarios, and spoofed ICMP traffic, considering indicators such as packet rate, Snort alerts, active sources, RTT, packet loss, and mitigation response.
6. To preprocess the selected multi-stage network-traffic dataset and generate a PCA-based feature representation suitable for subsequent attack analysis.
7. To segment broad attack families into detailed behavioral subpatterns and evaluate the stacking-ensemble classification approach adopted from the reference detection methodology.
8. To develop an improved hierarchical CatBoost mechanism that first predicts the attack family and then identifies the corresponding behavioral subpattern.
9. To generate a joint confidence score from family-level and subpattern-level probabilities and use stage-transition information to represent changes in multi-stage attack behavior.
10. To transfer the predicted attack family, subpattern, and confidence information through the detection–game bridge and integrate this evidence with the existing Game Theory and SDN mitigation process.
11. To evaluate the Artificial Intelligence and integrated defense components using classification metrics, confidence analysis, Game Theory decisions, SDN action mapping, and OpenFlow rule generation.

1.6 Literature Basis

The project is supported by complementary studies covering both the original SDN security framework and the current multi-stage Artificial Intelligence extension.

The original project phase was mainly supported by studies [1]–[7]. The SEC-SDN study [1] provided the closest foundation for integrating SDN, Snort-based intrusion detection, Game Theory, and adaptive DDoS mitigation. Other studies supported practical SDN monitoring and controller evaluation [2], the need to integrate detection with active mitigation [3], [4], resource-aware security design [5], and attacker–defender modeling using adaptive or dynamic Game Theory [6], [7]. Together, these studies supported the implementation of the original monitoring, detection, decision, and OpenFlow-based mitigation workflow.

The primary methodological reference for the current Artificial Intelligence extension is the study by Gong, Cho, and Choi [8], *Detection of Multi-Stage Attacks Through Attack Pattern Segmentation*. The study explains that advanced attacks may contain multiple behavioral patterns, some of which resemble benign traffic or overlap statistically with other attacks. To improve their detection, the authors proposed clustering-based attack pattern segmentation to divide broad attack classes into detailed subpatterns, followed by a stacking ensemble for classification. The method was evaluated using the CIC-IDS-2018 dataset and achieved an average F1 score of 0.9732 for the evaluated multi-stage attack patterns.

The present project adopts the main segmentation concept from [8] and extends the detection stage further. After preprocessing and PCA-based feature transformation, the selected traffic families are segmented into behavioral subpatterns. In addition to evaluating the stacking-based approach, a hierarchical CatBoost mechanism is developed to first identify the attack family and then classify the corresponding subpattern. Family-level and subpattern-level probabilities are also combined to generate a joint confidence score.

The second main study supporting the current phase is the work by Kour, Karim, and Dersin [9], *Modelling Cybersecurity Strategies with Game Theory and Cyber Kill Chain*. This study models cybersecurity as a non-cooperative interaction between an attacker and a defender across different stages of a cyberattack. It supports the project’s use of stage-related attack

evidence, attacker–defender reasoning, payoff evaluation, and adaptive defense selection.

Therefore, studies [1]–[7] preserve the theoretical and practical basis of the original SDN defense framework, while studies [8] and [9] provide the main methodological support for the current multi-stage detection and stage-aware defense extension.

Table 1.1 Summary of the adopted studies supporting the project

Ref.	Study title	Year	Main focus	Contribution to the project
[1]	Enhancing network security through integration of game theory in software-defined networking framework	2025	SEC-SDN framework integrating SDN, Snort, and game theory for adaptive DDoS mitigation	Primary reference for the original SDN security and adaptive mitigation framework.
[2]	A novel sFlow based DDoS detection model in software defined networking	2024	Early DDoS detection in SDN controllers using sFlow with practical controller-based evaluation	Supports the practical monitoring and SDN-controller detection perspective
[3]	Software-Defined Networking Security Detection Strategies and Their Limitations with a Focus on Distributed Denial-of-Service for Small to Medium-Sized Enterprises	2025	Systematic review of SDN-based DDoS detection and mitigation with emphasis on lightweight solutions	Strongly supports the research gap and the need for practical integrated solutions
[4]	Hybrid Approach for Detection and Mitigation of DDoS Attacks Using Multi-feature Selection, Unsupervised Learning, and Game Theory	2025	Adaptive detection and mitigation using feature selection, clustering, and game theory	Supports the detect-to-mitigate logic and adaptive strategic defense
[5]	Resource-Aware Ensemble Framework for DDoS Detection in SDN via Novel Game-Theoretic Feature Selection	2026	Resource-aware SDN DDoS detection with game-theoretic feature selection	Supports efficiency, deployment feasibility, and low-overhead design
[6]	Detecting malicious nodes using game theory and reinforcement learning in software-defined networks	2025	Game-theoretic and learning-based modeling of adversarial network behavior	Supports attacker–defender modeling and adaptive reasoning in SDN security
[7]	Advanced EDoS eye: A pragmatic model to mitigate economic denial of sustainability attack in cloud	2026	Dynamic game-based decision module for adaptive defense and threshold selection	Supports the theoretical argument for dynamic game-driven response logic

Ref.	Study title	Year	Main focus	Contribution to the project
	system applying dynamic game-based decision module			
[8]	Detection of Multi-Stage Attacks Through Attack Pattern Segmentation	2025	Clustering-based attack pattern segmentation and stacking ensemble classification for multi-stage attacks	Primary methodological reference for the current attack-pattern segmentation and AI detection extension.
[9]	Modelling Cybersecurity Strategies with Game Theory and Cyber Kill Chain	2025	Non-cooperative mixed-strategy modeling of attacker–defender interaction across Cyber Kill Chain stages	Main theoretical reference for stage-aware attacker–defender modeling and strategic defense selection.

1.7 Research Gap

The original project addressed a gap in SDN security research where attack detection and mitigation are often treated as separate processes. Several studies focus primarily on identifying DDoS traffic, while practical integration between traffic inspection, controller-side decision making, and active mitigation remains limited [3], [4]. In addition, practical SDN security mechanisms must consider controller overhead and deployment feasibility rather than classification accuracy alone [3], [5].

To address this gap, the original project established a closed-loop SDN workflow in which traffic is mirrored and inspected by Snort, security alerts are forwarded to the Ryu Controller, Game Theory is used to evaluate the observed attack context, and OpenFlow rules are applied to enforce graduated mitigation.

The current project phase addresses an additional gap related to multi-stage attack representation and detection. Broad class labels may hide several statistically distinct behaviors within the same attack family, particularly when some attack stages resemble benign traffic or overlap with other attack patterns [8].

The attack pattern segmentation approach proposed in [8] addresses this detection problem by generating and classifying detailed subpatterns. However, its output is not connected to an operational SDN defense framework that performs controller-side stage tracking, Game Theory-based defense selection, and OpenFlow enforcement.

Conversely, the existing SDN and Game Theory components provide adaptive decision making and mitigation but do not identify both the general attack family and its detailed behavioral subpattern.

The current extension bridges these two directions. Attack pattern segmentation and hierarchical CatBoost classification provide the predicted family, subpattern, and joint confidence score, which are transferred through the detection–game bridge and incorporated into the existing multi-stage tracking and Game Theory-based SDN defense process.

1.8 Challenges

The original SDN framework involved several practical challenges, including processing latency, false-positive alerts, controller-related resource overhead, and the need to select a proportional defense strategy rather than applying the strongest mitigation action to every suspicious event [1], [3], [4], [5].

The current extension introduces additional challenges related to multi-stage attack detection. Different behaviors within the same attack family may overlap statistically with benign or other attack patterns, making subpattern generation and classification more difficult [8]. Class imbalance and rare subpatterns may also reduce the reliability of model training and evaluation.

Hierarchical classification introduces another dependency because an incorrect family-level prediction may direct a sample to an unsuitable family-specific subpattern model. For this reason, family confidence is evaluated and combined with subpattern confidence to produce a joint confidence score.

A final challenge is integrating the Artificial Intelligence output with the existing SDN

framework. The predicted attack family, subpattern, and confidence information must be transferred consistently through the detection–game bridge while preserving the existing stage-tracking, Game Theory decision, and OpenFlow enforcement processes.

1.9 Practical Applications

The original framework provides an experimental platform for studying adaptive DDoS and spoofing defense in SDN environments using Mininet, Open vSwitch, Ryu, Snort, Game Theory, and OpenFlow-based mitigation.

The current Artificial Intelligence extension broadens this platform toward the analysis of multi-stage attacks. Instead of producing only a general malicious or benign indication, the developed detection pipeline identifies the predicted attack family, its behavioral subpattern, and the associated confidence level.

Although the project is implemented in an academic emulated environment rather than a production network, it provides a proof-of-concept for connecting detailed data-driven attack detection with programmable and adaptive network defense.

1.10 Chapter Summary

This chapter presented the background, problem, motivation, objectives, literature basis, research gap, and main challenges of the project. The original work established an SDN security framework integrating Snort-based detection, Ryu-based monitoring, Game Theory decision making, and OpenFlow mitigation.

The current phase extends this foundation with multi-stage attack analysis, PCA-based data representation, attack pattern segmentation, hierarchical CatBoost classification, and confidence-aware detection. The resulting AI evidence is integrated with the existing stage-aware Game Theory defense process, creating a more detailed detection-to-mitigation workflow.

2 Chapter Two: Theoretical Framework

This chapter presents the theoretical foundation of the original and extended phases of the project. The original phase is summarized through the main concepts required to understand the implemented SDN security framework, including centralized control, OpenFlow-based enforcement, intrusion detection, and Game Theory-based defense selection.

The chapter then focuses on the concepts supporting the current phase, including multi-stage cyberattacks, Markov-based stage modeling, network-traffic feature engineering, Principal Component Analysis, attack pattern segmentation, ensemble learning, hierarchical classification, and prediction-confidence analysis. These concepts provide the theoretical basis for detailed attack identification and its integration with the existing SDN defense workflow.

2.1 Software-Defined Networking and Security Architecture

Software-Defined Networking (SDN) separates the control plane from the data plane and places network-control logic within a centralized programmable controller. The architecture is commonly organized into application, control, and infrastructure layers, enabling forwarding behavior and security policies to be managed dynamically [1], [2].

In the implemented framework, Open vSwitch operates in the infrastructure layer, while the Ryu Controller manages network decisions and mitigation. Traffic is mirrored toward Snort for inspection, allowing monitoring, detection, and controller-based response to operate as one coordinated SDN security workflow [1], [3].

The basic SDN architecture supporting the project is illustrated in Figure 2.1.

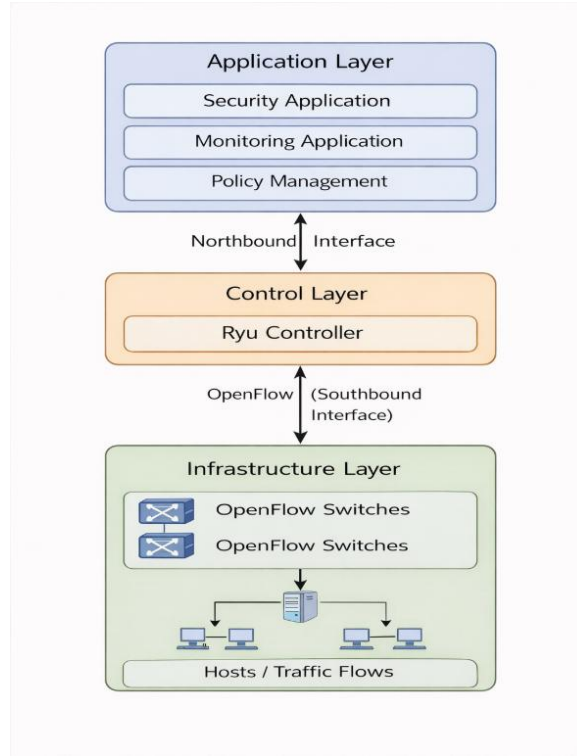


Figure 2.1 Basic SDN architecture used as the theoretical basis of the project, adapted from [1].

2.2 OpenFlow-Based Monitoring and Enforcement

OpenFlow provides the communication mechanism between the SDN controller and programmable switches, allowing the controller to install, modify, and remove forwarding rules dynamically [1], [3]. In the implemented framework, Open vSwitch also mirrors selected traffic toward Snort for inspection without changing the original forwarding path.

Defense actions are enforced through normal forwarding, OpenFlow meters for graduated rate limiting, and high-priority drop rules for severe malicious traffic. Thus, OpenFlow connects the controller's defense decision with the actual network response.

2.3 Intrusion Detection and SDN Security Integration

The Ryu controller represents the central decision component of the implemented SDN framework. It maintains network-flow information, receives security evidence, evaluates the observed traffic state, and applies the selected mitigation action through OpenFlow. Because

centralized controllers may be affected by excessive Packet-In requests and flooding traffic, controller-aware detection and mitigation are essential in SDN security [1], [3], [5].

The original project phase focused on DDoS, ICMP flooding, and IP-spoofing behavior. Snort inspected mirrored traffic and generated alerts when predefined suspicious patterns were detected. A forwarding agent then transferred normalized alert information from the Snort virtual machine to the Ryu controller, separating traffic inspection from controller-side decision making [1].

This design forms the detection and control foundation retained in the extended project. Snort alerts and controller-observed traffic indicators continue to support the original security workflow, while the current Artificial Intelligence component provides more detailed identification of multi-stage attack families and behavioral subpatterns. The theoretical basis of this AI component is presented in the subsequent sections.

2.4 Game Theory and Attacker-Defender Interaction

Game Theory models the strategic interaction between an attacker and a defender whose decisions affect one another. In cybersecurity, the defender must balance attack mitigation with the cost of unnecessarily restricting legitimate traffic [1], [4], [7].

In the implemented framework, the attacker is represented by one or more hosts generating suspicious traffic, while the defender is represented by the Ryu Controller supported by the dynamic game engine. According to the observed network condition, the defender selects among ALLOW, RL_1, RL_2, RL_3, and BLOCK.

The current phase extends this interaction by considering repeated observations, attack-stage changes, Snort evidence, spoofing information, attack intensity, active sources, and Artificial Intelligence evidence. This stage-aware formulation is supported by the attacker–defender perspective presented in [9].

Table 2.1 summarizes the mapping between the main Game Theory elements and their implementation in the project.

Table 2.1 Game-Theoretic Mapping Used in the Proposed Project.

Game Element	Project Mapping
Attacker	A host or group of hosts generating suspicious or malicious network traffic
Defender	The Ryu controller supported by the dynamic game engine
Observed attack states	NORMAL, PROBE, FLOOD_LOW, FLOOD_HIGH, IP_SPOOFING, and SPOOFED_FLOOD
Defender Strategies	ALLOW, RL_1, RL_2, RL_3, and BLOCK.
Main Observations	Packet rate, Snort alert, spoofing information, packet loss, RTT, active sources, current stage, and previous stage
Defender Objective	Reduce attack impact while preserving legitimate traffic and avoiding unnecessary mitigation
Attacker Objective	Increase disruption, conceal malicious behavior, or avoid effective mitigation
Decision Logic	Evaluate the observed state, transition behavior, and strategy payoffs before selecting the defense action

2.5 Multi-Stage Cyberattacks

A multi-stage cyberattack consists of a sequence of related behaviors that develop over time rather than appearing as one isolated malicious event. Different stages may use different attack vectors, tools, traffic characteristics, or levels of intensity. Consequently, representing the complete attack using one general label may hide the detailed behaviors that compose the attack scenario and make detection more difficult [8].

The Cyber Kill Chain provides a stage-oriented view of cyberattacks by describing how attacker behavior progresses through a sequence of activities. When combined with Game Theory, these stages can also be used to analyze how attacker and defender strategies change during the attack process [9]. In this project, the same general principle is used to distinguish changes in observed network behavior and support an adaptive response rather than treating all suspicious traffic as one fixed attack state.

The implemented framework represents the observed behavior through the states NORMAL, PROBE, FLOOD_LOW, FLOOD_HIGH, IP_SPOOFING, and SPOOFED_FLOOD. These states describe the traffic condition observed by the controller; they are not presented as the formal stages of the Cyber Kill Chain. In the implementation, these operational states are used to represent changes in traffic behavior and severity over successive controller observations, while the Cyber Kill Chain is used only as a conceptual stage-based reference for understanding attack progression.

The controller maintains a history for each monitored destination and compares the previous state with the current state to identify meaningful transitions, including changes between flooding and spoofing-related behavior.

Multi-stage evidence is then used together with traffic indicators, Snort alerts, active-source information, and Artificial Intelligence output. This allows the framework to distinguish an isolated observation from an evolving attack sequence and provides additional context for the subsequent Markov and Game Theory decision processes.

The relationship between the progression of multi-stage attack behavior and the corresponding security-state tracking process is illustrated in Figure 2.2.

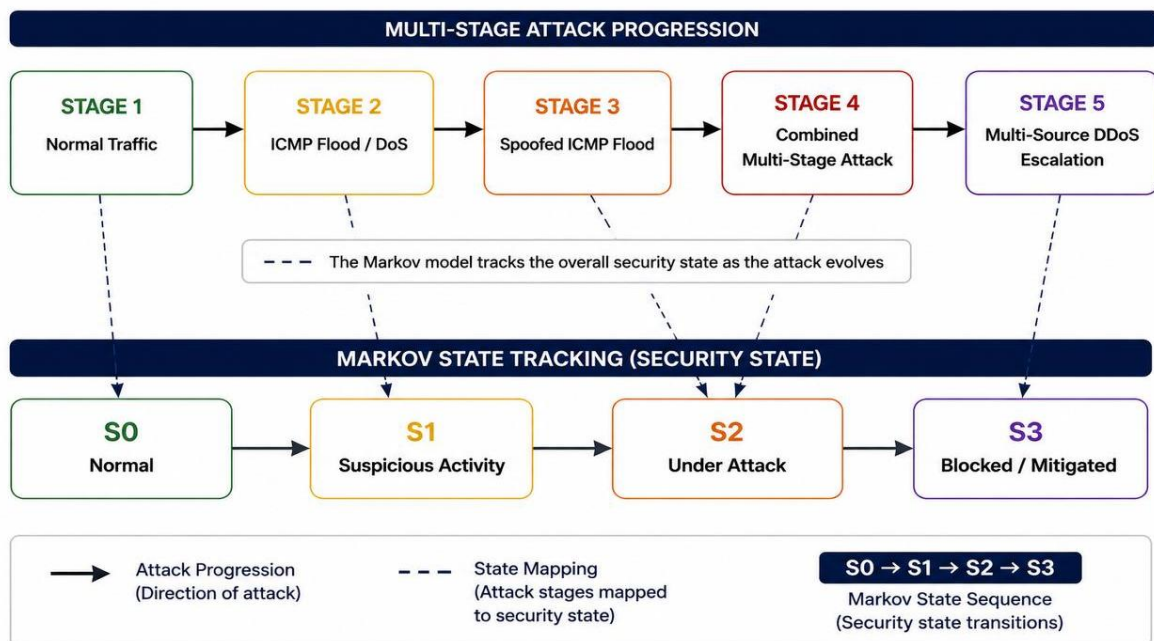


Figure 2.2 Illustrative relationship between multi-stage attack progression and Markov-based security-state tracking in the project.

2.6 Markov-Based Attack Stage Modeling

A Markov model represents a dynamic system through a set of states and the probabilities of transition between them. Its basic assumption is that the probability of the next state depends on the current state. This makes it suitable for analyzing network behavior that changes across successive observation rounds rather than evaluating each traffic event independently.

The transition from a current state S_i to a subsequent state S_j can be represented as [22]:

$$P(X_{t+1} = S_j \mid X_t = S_i)$$

where X_t denotes the security state at time t , and P_{ij} denotes the probability of moving from state S_i to state S_j . Depending on the observed evidence, the system may remain in its current state, move toward a more severe state, or return to a less severe state.

As illustrated previously in Figure 2.2, the progression of multi-stage attack behavior is mapped to general security states that represent normal traffic, suspicious activity, a detected attack, and blocked or mitigated traffic. This simplified representation explains the relationship between attack progression and temporal state tracking. It does not imply that every attack must pass through all states in one fixed sequence.

In the operational implementation, the dynamic game engine uses the more detailed traffic states NORMAL, PROBE, FLOOD_LOW, FLOOD_HIGH, IP_SPOOFING, and SPOOFED_FLOOD. These states describe the detected behavior and its severity more precisely than the general conceptual states shown in Figure 2.2. The transition matrix assigns probabilities to movement between the operational states, including the probability of remaining in the same state.

During each decision round, the framework evaluates the previous stage together with the current traffic evidence, including traffic intensity, spoofing indicators, Snort alerts, active-source information, and available Artificial Intelligence output. The Markov result provides temporal context about the evolving attack condition to the Game Theory engine. Markov

modeling therefore supports stage tracking and state estimation, while Game Theory remains responsible for selecting the appropriate defense action.

2.7 Artificial Intelligence for Network Intrusion Detection

Artificial Intelligence-based intrusion detection uses data-driven models to learn the statistical characteristics of normal and malicious network traffic. Unlike signature-based detection, which depends on predefined rules, machine learning models can classify traffic according to patterns extracted from the available features. This makes AI useful for analyzing large network-flow datasets and identifying complex behaviors that may not be represented by one fixed signature.

However, detecting multi-stage attacks remains difficult because different attack behaviors may have similar statistical distributions, and some malicious patterns may resemble normal traffic. In addition, assigning one broad label to all traffic associated with an attack campaign may hide the detailed behaviors that form the attack. These limitations can reduce the ability of a classifier to separate normal traffic from complex attack patterns accurately [8].

The AI component developed in this project addresses this problem through a multi-stage processing pipeline. The selected network-traffic data is cleaned and transformed, informative features are prepared, and Principal Component Analysis is applied before attack pattern segmentation. The resulting subpatterns are then used for ensemble-based and hierarchical classification. The final detection output includes the predicted attack family, the identified subpattern, and the associated prediction confidence.

The AI component does not replace the existing Snort and Ryu-based security framework. Instead, it provides a more detailed data-driven analysis that complements the original detection indicators. Its output is subsequently transferred to the existing defense process, where Markov-based stage information and Game Theory are used to support the selection of an appropriate mitigation action.

2.8 Network-Traffic Feature Engineering

Network-traffic feature engineering transforms raw or flow-based observations into numerical characteristics that are more suitable for machine learning. This step is important because the original flow representation may lose temporal and behavioral information when a complex attack campaign is summarized into independent traffic records [8].

Port information is particularly useful in multi-stage attack analysis because changes in source and destination port usage can reflect the services, tools, and activities involved in different attack behaviors. However, individual port numbers are highly localized and may not be directly suitable as machine-learning inputs. Therefore, statistical properties of port behavior can provide a more general representation than the raw values themselves [8].

The reference study calculates port variance over sliding windows to capture changes in network behavior at different observation scales. Smaller windows describe short-term fluctuations, while larger windows represent broader behavioral characteristics. The resulting variance values are added to the traffic-flow dataset as engineered features to support the separation of normal and multi-stage attack patterns [8].

The effect of port-variance feature extraction over a larger sliding window is illustrated in Figure 2.3.

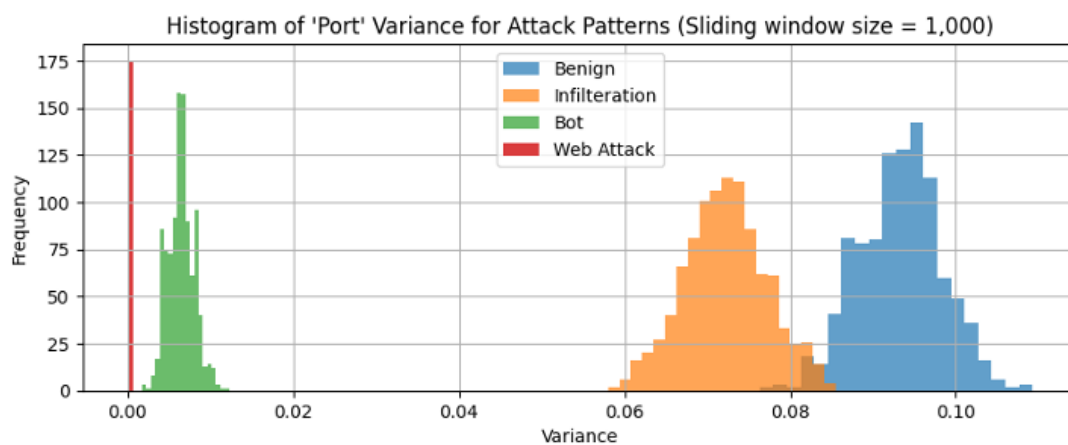


Figure 2.3 Port-Variance Distributions for Benign and Multi-Stage Attack Patterns Using a Sliding Window of Size 1,000, Adapted from [8]

In the implemented project pipeline, port-variance features are generated after data cleaning and before dimensionality reduction. This preserves additional behavioral information that

may help distinguish Benign, Bot, Web Attacks, and Infiltration traffic before the data is standardized and transformed using Principal Component Analysis.

2.9 Principal Component Analysis

Network-traffic datasets commonly contain a large number of numerical features, some of which may be correlated or provide redundant information. Processing all available features can increase computational cost and may make the subsequent clustering and classification stages more difficult. Principal Component Analysis (PCA) addresses this issue by transforming the original features into a smaller set of uncorrelated variables called principal components.

Each principal component represents a linear combination of the original features. The first component preserves the largest possible amount of variance in the data, while each subsequent component preserves the greatest remaining variance under the condition that it is uncorrelated with the preceding components. Therefore, PCA reduces dimensionality while retaining the main statistical structure of the dataset.

The reference study applies PCA before clustering because packet- and flow-based datasets may contain redundant statistical information. Dimensionality reduction improves the efficiency of the attack pattern segmentation process and provides a compact representation for distinguishing detailed attack behaviors [8].

In the implemented project, the engineered numerical features are first standardized using StandardScaler. Incremental PCA is then fitted in batches so that the large compressed dataset can be processed without loading all records into memory simultaneously. A maximum of 30 principal components is considered during fitting, and the cumulative explained variance is evaluated to determine how many components preserve the required information.

The first 26 principal components are used in the subsequent attack pattern segmentation and classification stages because they retain at least 95% of the cumulative explained variance.

The transformed dataset therefore contains PC1 through PC26 together with the corresponding traffic label, providing a reduced and standardized representation for the following stages.

2.10 Attack Pattern Segmentation

Attack pattern segmentation is the main methodological contribution adopted from the reference study [8] and forms the foundation of the Artificial Intelligence detection pipeline implemented in this project. Instead of treating every attack family as a single homogeneous class, this approach divides each attack family into several behavioral subpatterns that share similar statistical characteristics. As a result, the classifier learns simpler and more consistent patterns rather than attempting to model highly diverse attack behaviors within one class [8].

Conventional machine learning classifiers often struggle with multi-stage attacks because different attack procedures may produce substantially different traffic distributions while still belonging to the same attack family. Consequently, assigning a single label to all traffic associated with an attack family may increase intra-class variation and reduce classification accuracy. Attack pattern segmentation addresses this limitation by grouping samples with similar behavioral characteristics before the classification stage [8].

The segmentation process is performed independently for each attack family after dimensionality reduction. Samples that exhibit similar statistical behavior are grouped together using clustering techniques, producing a set of representative subpatterns for each family. These subpatterns preserve the detailed behavioral structure of the original attack while reducing the variability observed within the complete attack family.

The overall segmentation-based detection methodology proposed in the reference study is illustrated in Figure 2.4.

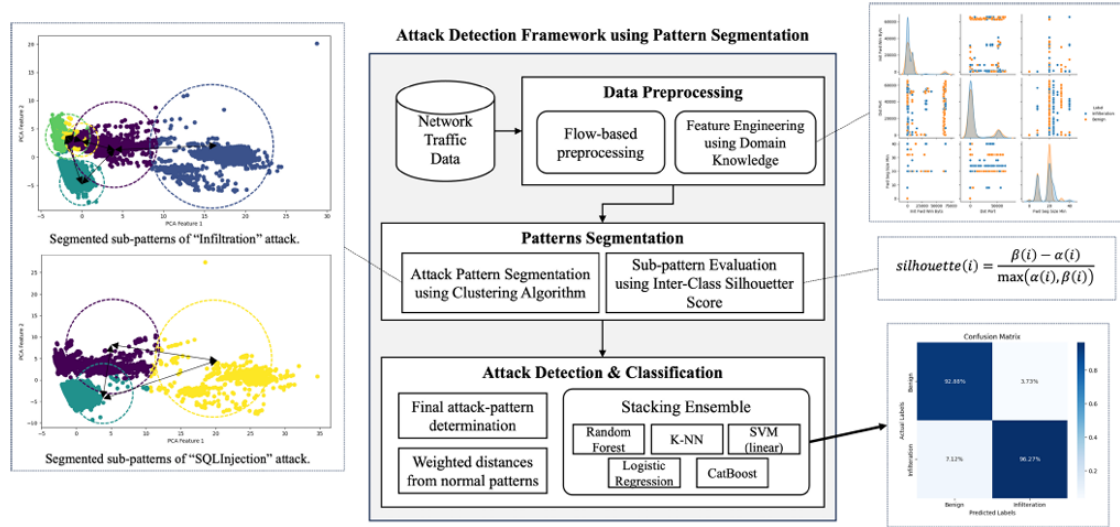


Figure 2.4 Overview of the Attack Pattern Segmentation-Based Network Attack Detection Technique, Adapted from [8]

In the implemented project, attack pattern segmentation is applied after PCA using class-specific clustering models. Separate clustering is performed for Benign, Bot, Web Attacks, and Infiltration, allowing each attack family to be divided into its own behavioral subpatterns. Each network-flow record is therefore assigned both an attack family label and a corresponding subpattern label, which are subsequently used during hierarchical classification.

2.11 Ensemble and Stacking Classification

Ensemble learning combines the predictions of multiple machine learning models to obtain a more reliable final decision. The main purpose is to benefit from the complementary strengths of different classifiers rather than depending on one model whose performance may vary across different traffic distributions.

Stacking is an ensemble method organized into two learning levels. At the first level, several base classifiers are trained on the same input features. Each classifier learns the data from a different perspective and produces prediction probabilities or intermediate predictions. At the second level, these outputs are provided to a meta-learner, which learns how to combine them and produce the final classification result.

The stacking framework presented in the reference study uses Random Forest, k-Nearest Neighbors, Logistic Regression, Linear Support Vector Machine, and CatBoost as base learners. These models were selected to capture linear, nonlinear, tree-based, and local-neighborhood relationships within the segmented attack data. A meta-learner then combines their outputs to determine the final subpattern and its corresponding parent attack pattern [8].

The two-level stacking classification process adopted as the initial AI detection mechanism is illustrated in Figure 2.5.

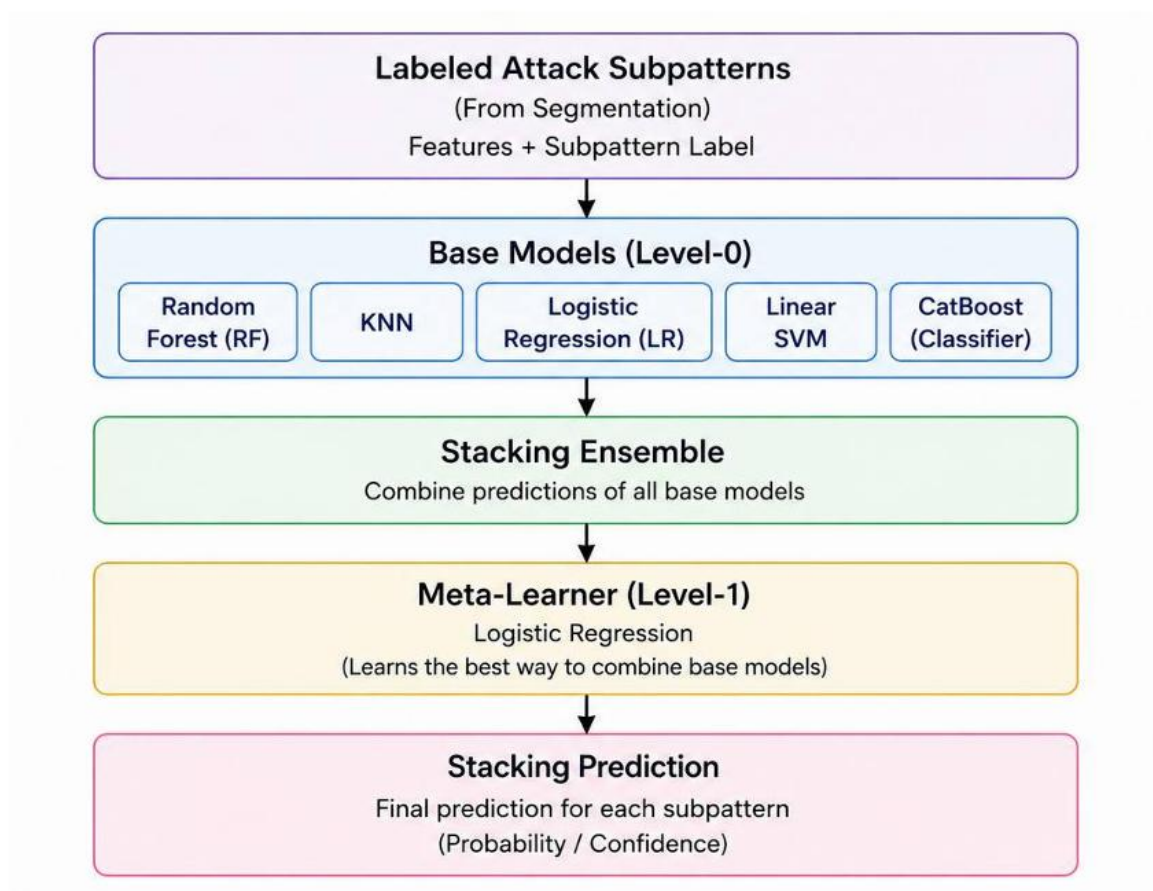


Figure 2.5 Two-Level Stacking Ensemble Used for Segmented Attack-Pattern Classification

In the project, stacking represents the initial Artificial Intelligence classification stage applied to the segmented dataset. The input consists of the principal-component features and the generated subpattern labels. The resulting model is evaluated to determine how effectively the segmented attack behaviors can be distinguished before developing the improved hierarchical classification mechanism presented in the following section.

This stacking stage serves as the reference-aligned baseline for segmented classification, while the hierarchical CatBoost mechanism presented next represents the improved classification stage developed in the project.

2.12 Hierarchical Attack Classification

Hierarchical classification organizes the prediction task into multiple related levels rather than attempting to identify every detailed class through one flat classifier. This structure is suitable when the target classes have a natural parent–child relationship. In this project, each behavioral subpattern belongs to one of four parent traffic families: Benign, Bot, Web Attacks, or Infiltration.

The improved hierarchical classifier operates through two prediction levels. At Level 1, a CatBoost model receives the selected principal-component features and predicts the general traffic family. At Level 2, the predicted family determines which family-specific CatBoost model is used to identify the detailed subpattern. Therefore, the second-level model distinguishes only between the subpatterns belonging to the selected family rather than comparing all subpatterns simultaneously.

This organization reduces the complexity of the detailed prediction task and preserves the relationship between each subpattern and its parent family. It also allows the training configuration and class-balancing strategy to be adjusted according to the characteristics of each family. Separate second-level models are therefore trained for Benign, Bot, Web Attacks, and Infiltration traffic.

The final hierarchical output contains both the predicted attack family and the corresponding behavioral subpattern. The probability produced at each level is retained so that the system can evaluate the reliability of the complete hierarchical prediction.

2.13 Prediction Confidence and Joint Scoring

Prediction confidence indicates how strongly a classification model supports its selected class. In the hierarchical classifier, confidence is produced at two related levels. The Level-1 model provides the probability of the predicted attack family, while the selected Level-2 model provides the probability of the predicted subpattern within that family.

In the implemented hierarchical classifier, the two prediction probabilities are combined using the following project-defined joint score:

$$\text{Joint Score} = \text{Family Probability} \times \text{Subpattern Probability}$$

The joint score represents the confidence of the complete family–subpattern prediction path. A high value indicates that both the parent family and the detailed subpattern are supported by their corresponding models. Conversely, a lower value may result from uncertainty at either classification level.

In the implemented classifier, a family-confidence threshold of 0.90 is used. When the highest family probability is equal to or greater than this threshold, the corresponding family-specific model is used to generate the final subpattern prediction. When the family confidence is below 0.90, the two most probable family candidates are considered. Their corresponding subpattern predictions and joint scores are calculated, and the family–subpattern combination with the higher joint score is selected.

The final output therefore includes the predicted attack family, the predicted subpattern, and the joint score as the overall confidence value. Low-confidence predictions are also recorded during evaluation because they may indicate overlapping traffic distributions, weak subpatterns, or uncertainty between closely related attack behaviors.

2.14 Integration of AI Detection with SDN Defense

The Artificial Intelligence component extends the original SDN security framework by providing a more detailed description of the detected traffic. Instead of producing only a general malicious or benign decision, the hierarchical classifier identifies the predicted attack family, the corresponding behavioral subpattern, and the joint confidence score.

The detection result is prepared by the integration bridge and transmitted through an HTTP POST request to the receiver deployed in the VMware environment. The receiver validates the incoming information and forwards the accepted detection result to the dynamic Game Theory engine. This communication mechanism allows the AI analysis and the SDN defense components to operate as separate modules while exchanging the information required for adaptive decision-making.

The end-to-end integration between AI-based detection and SDN-based defense is illustrated in Figure 2.6.

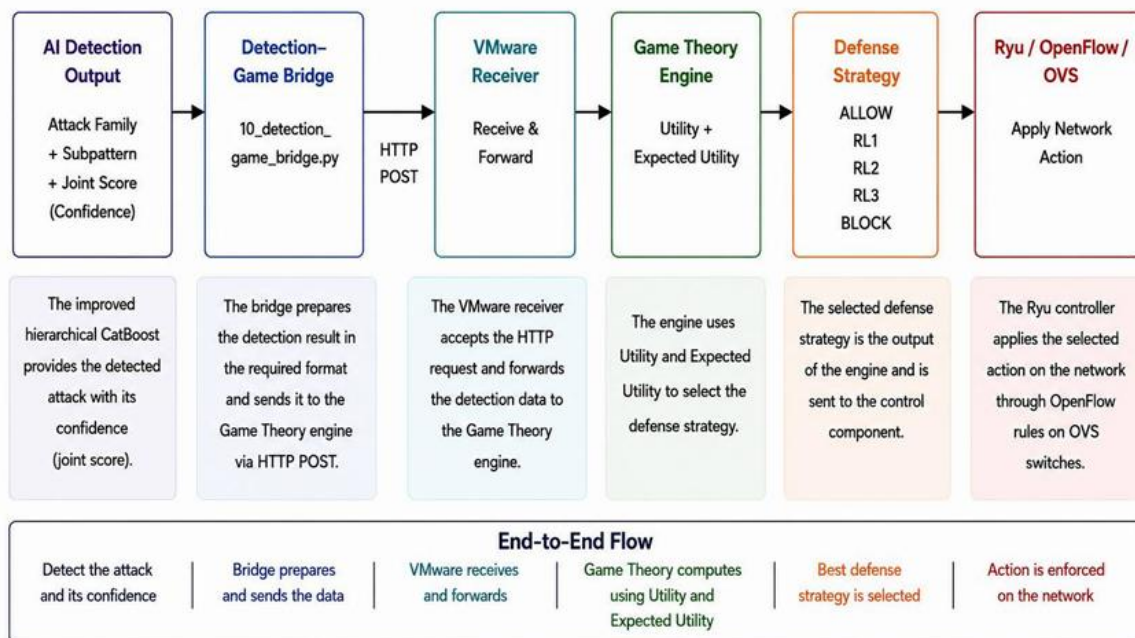


Figure 2.6 End-to-End Integration of AI Detection, Game Theory, and SDN-Based Defense

The Game Theory engine combines the AI result with the available network observations, including traffic intensity, Snort alerts, packet loss, RTT, spoofing indicators, active-source information, the current attack state, and the previous state. The AI result is accepted only when an attack is detected and the joint confidence satisfies the configured acceptance condition. Therefore, the machine-learning output is treated as an additional source of evidence rather than the only basis for the defense decision.

After evaluating the possible defense strategies, the engine selects one of the available actions: ALLOW, RL1, RL2, RL3, or BLOCK. The selected action is then delivered to the

Ryu controller, which translates the decision into OpenFlow rules and applies it through the Open vSwitch infrastructure. This completes the end-to-end process from traffic analysis and attack identification to adaptive network enforcement.

2.15 Chapter Summary

This chapter presented the theoretical foundation of the project, including SDN, OpenFlow, intrusion detection, Game Theory, and Markov-based state tracking. It also introduced the Artificial Intelligence concepts used in the extended phase, including feature engineering, PCA, attack pattern segmentation, stacking, hierarchical classification, and confidence scoring.

The chapter further explained how the AI detection output is integrated with the existing SDN defense framework to support adaptive response through Game Theory, Ryu, OpenFlow, and Open vSwitch. The next chapter presents the implementation environment, dataset, system components, and methodology.

3 Chapter Three: Working Environment

3.1 Introduction

This chapter presents the working environment and implementation methodology of the complete security framework. The original SDN environment, including Mininet, Open vSwitch, Ryu, Snort, Game Theory, and OpenFlow-based mitigation, is summarized as the foundation of the implemented system.

The framework was first extended with multi-stage attack tracking using operational security states, previous and current stage information, and Markov-based transition logic to represent the evolution of suspicious behavior across successive decision rounds.

Artificial Intelligence was subsequently introduced as a second extension. The AI pipeline performs data preparation, feature engineering, PCA, attack pattern segmentation, ensemble classification, and hierarchical CatBoost-based prediction to identify the attack family, behavioral subpattern, and associated confidence.

Finally, the AI detection output is integrated with the existing multi-stage, Game Theory, and SDN defense mechanisms to support adaptive decision making and programmable network enforcement.

3.2 Overall Project Implementation Environment

The complete project was implemented as two connected environments. The first environment represents the original Software-Defined Networking security platform and includes the Mininet network emulator, Open vSwitch, the Ryu controller, Snort IDS, the alert-forwarding mechanism, and the dynamic Game Theory engine. This environment is responsible for network emulation, traffic monitoring, threat evaluation, defense selection, and OpenFlow-based enforcement.

The second environment contains the Artificial Intelligence processing and classification pipeline. It is responsible for preparing the CIC-IDS-2018 traffic data, generating engineered features, applying dimensionality reduction, segmenting attack patterns, training the classification models, and producing the final attack-family, subpattern, and confidence output.

The complete implementation environment and the connection between the AI processing pipeline and the SDN defense platform are illustrated in Figure 3.1.

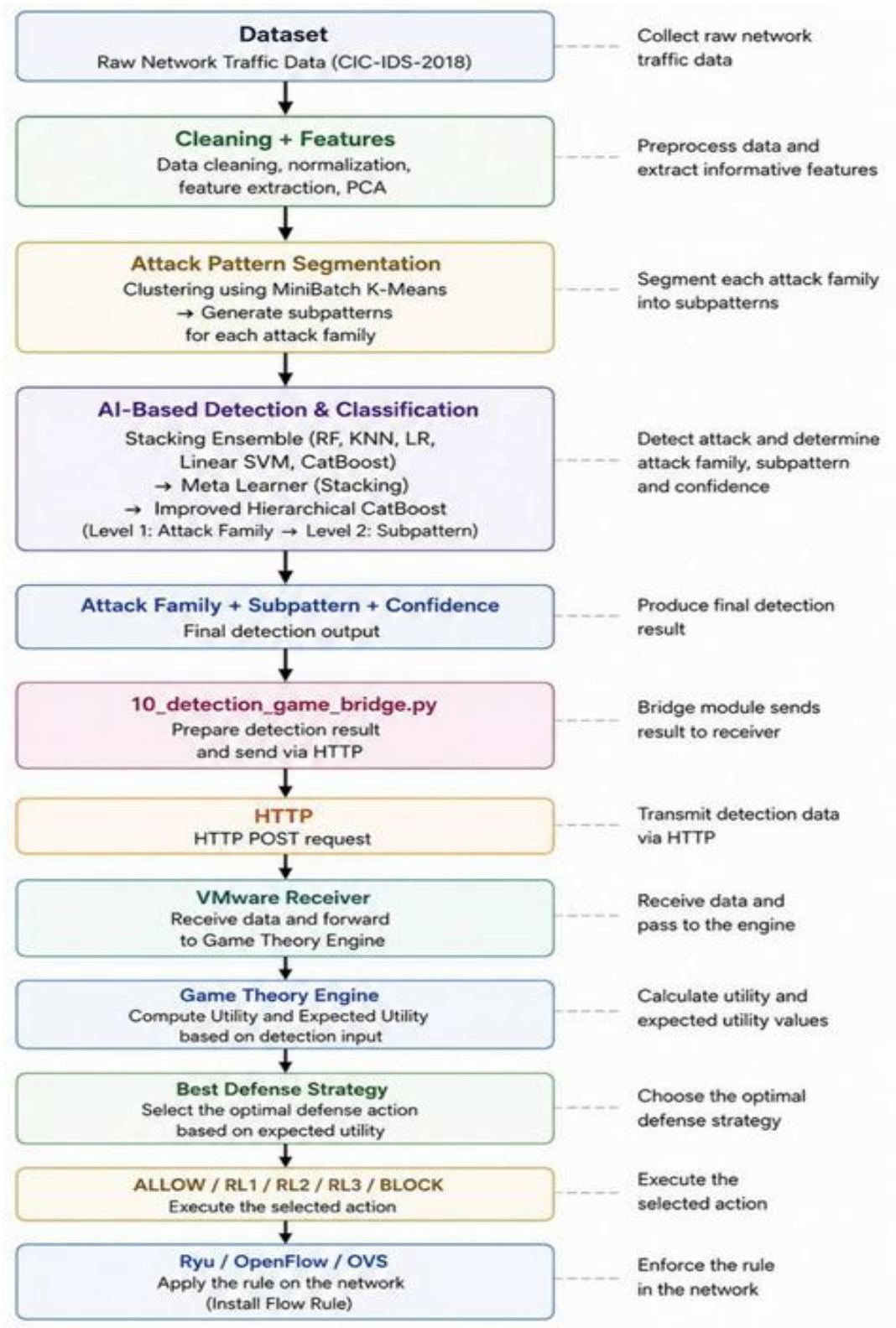


Figure 3.1 Complete AI-Driven Multi-Stage Attack Detection and SDN Defense Environment

The two environments are connected through an integration mechanism that transfers accepted AI detection results to the dynamic defense system. The Game Theory engine combines this information with the observations collected from the SDN environment, after which the selected defense action is enforced by the Ryu controller through OpenFlow rules and Open vSwitch.

3.3 Software and Development Components

The project was developed using a combination of virtualization, Software-Defined Networking, intrusion-detection, data-processing, and machine-learning technologies. The original environment provides network emulation, traffic inspection, controller-side decision-making, and OpenFlow enforcement, while the added Artificial Intelligence environment performs dataset preparation, feature transformation, attack pattern segmentation, model training, and prediction generation.

Python is the main development language shared by the two environments. It is used for the Ryu controller applications, alert-forwarding and integration scripts, Game Theory logic, data-processing stages, and machine-learning models. Table 3.1 summarizes the main software components and their roles in the complete project.

Table 3.1 Software and Development Components Used in the Complete Project

Role in the Project	Component / Technology	Environment
Hosts the virtual machines used for the SDN, monitoring, and integration environments	VMware Workstation	Virtualization
Provides the Linux environment used by the Mininet, Snort, and Ryu components	Ubuntu 22.04.5 LTS	Operating System
Implements the controller, agents, Game Theory logic, data processing, integration, and machine-learning scripts	Python 3.10.12	Development Language

Role in the Project	Component / Technology	Environment
Creates the emulated network topology containing hosts and OpenFlow switches	Mininet 2.3.0	SDN Environment
Performs packet forwarding, traffic mirroring, meter-based rate limiting, and flow-rule enforcement	Open vSwitch 2.17.9	Data Plane
Connects the Ryu controller to Open vSwitch and supports flow and meter operations	OpenFlow 1.3	Southbound Protocol
Collects network observations and applies the selected mitigation actions	Ryu 4.34	SDN Controller
Inspects mirrored traffic and generates alerts for suspicious behavior	Snort 3.1.5.0	Intrusion Detection
Generates ICMP flooding traffic during practical attack experiments	hping3 3.0.0-alpha-2	Attack Generation
Generates customized and spoofed packets for security-testing scenarios	Scapy 2.4.4	Packet Crafting
Loads, cleans, filters, transforms, and stores the network-traffic datasets	pandas	Data Processing
Supports numerical conversion, array operations, and invalid-value handling	NumPy	Numerical Processing
Provides data splitting, standardization, PCA, clustering, evaluation metrics, and supporting classifiers	scikit-learn	Machine Learning
Implements the family-level and family-specific hierarchical classification models	CatBoost	Gradient Boosting
Saves and loads scalers, PCA objects, clustering	joblib	Model Storage

Role in the Project	Component / Technology	Environment
models, and trained classifiers		
Transfers accepted AI detection results to the dynamic defense environment	HTTP-based Python bridge and receiver	AI–Defense Integration
Combines network and AI evidence and selects ALLOW, RL_1, RL_2, RL_3, or BLOCK	Dynamic Game Theory engine	Decision Module

The SDN-related versions in Table 3.1 were retained from the original implementation environment. The Artificial Intelligence libraries were added according to their actual functions in the project source code. Their installed package versions should be recorded directly from the final execution environment to preserve reproducibility and avoid reporting versions that were not used during model development.

The selected components support a modular implementation. Network emulation and live defense operate within the virtualized SDN environment, whereas computationally intensive dataset preparation and model training are executed through the AI pipeline. The saved preprocessing and model objects allow later stages to reuse the same transformations and classification logic without repeating the complete training process.

3.4 Original SDN Experimental Environment

The original project phase was implemented in a virtualized Software-Defined Networking environment. The environment combined Mininet, Open vSwitch, the Ryu controller, Snort IDS, an alert-forwarding agent, and a dynamic Game Theory engine. Its purpose was to monitor network traffic, detect suspicious behavior, evaluate the current threat condition, and apply an adaptive mitigation action.

Mininet was used to emulate the network hosts and switches, while Open vSwitch provided OpenFlow-based forwarding and traffic mirroring. Mirrored traffic was delivered to Snort for inspection, and generated alerts were forwarded to the controller environment. The Ryu controller then combined IDS evidence with network observations and transferred the

resulting state information to the Game Theory decision engine.

The available defense actions were ALLOW, RL_1, RL_2, RL_3, and BLOCK. The selected action was converted into OpenFlow flow rules or meter rules and applied to the corresponding switch. This original environment forms the defense platform that receives and uses the Artificial Intelligence detection output developed in the current project phase.

The main components of the original SDN security environment are illustrated in Figure 3.2.

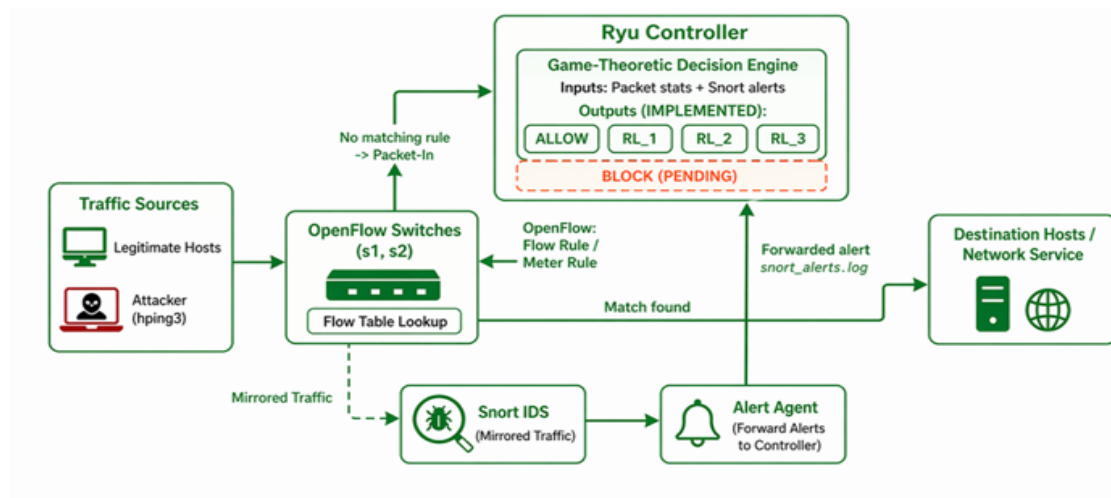


Figure 3.2 . Original SDN Monitoring, Detection, Decision, and Enforcement Environment

3.5 Virtual Machines and Network Interface Layout

The original SDN environment was distributed across separate virtual machines to isolate network emulation, traffic inspection, and controller-side processing. The Mininet virtual machine hosted the emulated topology, Open vSwitch instances, and the Ryu controller, while the Snort virtual machine was dedicated to inspecting mirrored network traffic.

Each virtual machine used a management interface for normal communication and additional interfaces for monitoring and traffic mirroring. On the Mininet virtual machine, separate interfaces were assigned to the mirrored outputs of the two OpenFlow switches. On the Snort virtual machine, dedicated monitoring interfaces received the copied traffic generated by the mirroring configuration.

This arrangement separated management communication from monitored traffic and allowed Snort to inspect traffic from both switches without interfering with the normal forwarding path.

The configured interfaces of the Mininet and Snort virtual machines are shown in Figures 3.3 and 3.4.

```

mininet@mininet-virtual-machine:~$ ip -br a
lo                UNKNOWN      127.0.0.1/8 ::1/128
ens33             UP          192.168.255.128/24 fe80::9239:a666:696f:c541/64
ens37             UP          192.168.57.129/24 fe80::40e3:1437:8ff9:e219/64
ens38             UP          192.168.57.130/24 fe80::1e62:e501:96e5:7564/64
ens39             UP          192.168.57.132/24 fe80::60df:8f6e:f6e2:cd74/64
mininet@mininet-virtual-machine:~$

```

Figure 3.3 Network Interfaces Configured on the Mininet Virtual Machine

```

user@ubuntu:~$ ip -br a
lo                UNKNOWN      127.0.0.1/8 ::1/128
ens33             UP          192.168.18.171/24 fe80::9cf2:940:4340:e579/64
ens36             UP          192.168.57.128/24 fe80::1cce:fdbf:252b:bdbe/64
ens38             UP          192.168.57.131/24 fe80::d8ad:984d:7b23:4de/64
ens39             UP          192.168.57.133/24 fe80::970c:be6e:fda8:4b0/64
user@ubuntu:~$

```

Figure 3.4 Management and Monitoring Interfaces Configured on the Snort Virtual Machine

Table 3.2 Virtual-Machine Interfaces Used for Management and Traffic Monitoring

VM	Interface	IPv4 Address	Purpose
Mininet VM	ens33	192.168.255.128/24	Management / Internet access
Mininet VM	ens37	192.168.57.129/24	Mirror output port for s1
Mininet VM	ens39	192.168.57.132/24	Mirror output port for s2
Snort VM	ens33	192.168.18.171/24	Management / SSH access
Snort VM	ens36	192.168.57.128/24	Monitoring interface for mirrored traffic

VM	Interface	IPv4 Address	Purpose
Snort VM	ens38	192.168.57.131/24	Monitoring interface for mirrored traffic

The dedicated monitoring interfaces support traffic visibility from both OpenFlow switches, while the management interfaces remain available for controller communication, remote access, and system administration. This arrangement separates management communication from monitored traffic and allows Snort to inspect traffic from both switches without interfering with the normal forwarding path.

3.6 SDN Topology and Security Components

The original SDN topology was implemented using two OpenFlow switches connected to a centralized Ryu controller. Six virtual hosts were distributed across the two switches, where hosts h1, h2, and h3 were connected to switch s1, while hosts h4, h5, and h6 were connected to switch s2. The hosts used the logical addresses 10.0.0.1 through 10.0.0.6 for h1 through h6, respectively. Thus, h1 (10.0.0.1) represented the main attacker host in the original scenario, while h4 (10.0.0.4) represented the primary victim. The monitoring and management interfaces connecting the virtual-machine components are summarized in Table 3.2.

The topology also included the main security components required for monitoring and mitigation. Open vSwitch mirrored traffic from both switches to the Snort virtual machine, while the alert-forwarding agent transferred the generated IDS alerts to the Ryu controller. The dynamic Game Theory engine then evaluated the network condition and selected an appropriate defensive action.

The topology and the interaction between the monitoring, detection, decision, and enforcement components are illustrated in Figure 3.5.

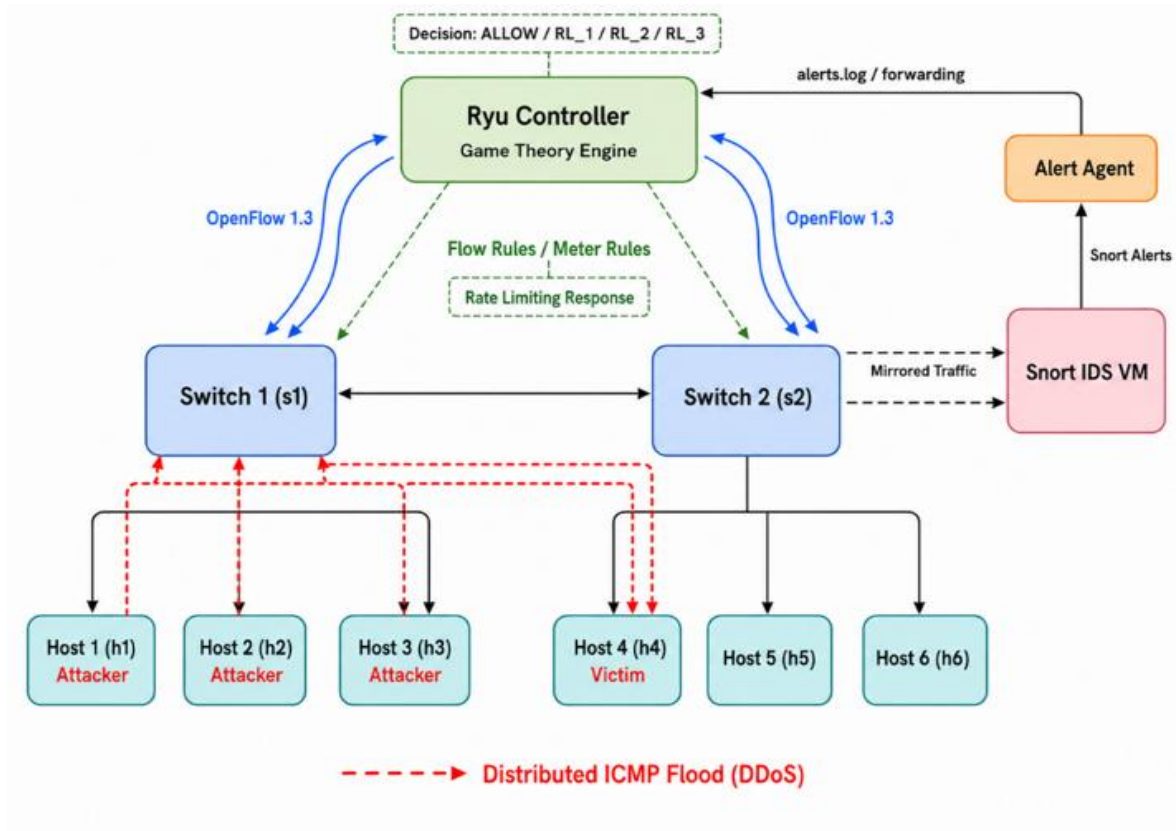


Figure 3.5 SDN Topology and Security-Component Integration

In single-source experiments, h1 was used as the primary attacker, whereas the distributed flooding scenario used multiple source hosts, as illustrated in Figure 3.5.

The available defense actions were ALLOW, RL_1, RL_2, RL_3, and BLOCK. These actions represent increasing mitigation severity. ALLOW is used when the observed traffic does not justify restriction. RL_1, RL_2, and RL_3 represent progressively stronger rate-limiting levels as the observed attack evidence and severity increase. BLOCK represents the strongest response and is reserved for traffic considered sufficiently severe or dangerous to require complete dropping. During practical evaluation, the selected response depends on the combined attack evidence rather than on the action name alone; the corresponding attack scenarios and resulting decisions are presented in Chapter Five.

An illustrative interpretation of these responses can be made according to the observed attack severity. Normal traffic may remain under ALLOW, while mild suspicious behavior may justify light rate limiting through RL_1. Stronger flooding evidence may lead to RL_2 or RL_3, whereas sufficiently severe or persistent malicious behavior may require BLOCK.

These examples are not fixed state-to-action rules; the final response is determined from the combined network evidence and the Game Theory decision process.

Rate-limiting decisions were enforced through OpenFlow meter rules, while severe or persistent malicious traffic could be blocked using a high-priority drop rule. This topology therefore provided the practical network environment required to evaluate adaptive attack detection and mitigation.

3.7 Artificial Intelligence Development Environment

The Artificial Intelligence extension was developed as an independent processing pipeline that complements the original SDN security platform. Unlike the SDN environment, which operates on live network observations and controller-side decisions, the AI environment focuses on dataset preparation, feature transformation, model training, evaluation, and prediction generation.

The implementation was developed using Python together with Pandas, NumPy, Scikit-learn, CatBoost, and Joblib. These tools were used for reading compressed datasets, cleaning and transforming features, applying Principal Component Analysis, segmenting attack patterns, training classification models, evaluating predictions, and saving the trained processing objects.

The AI environment was organized as a sequence of executable Python modules. Each module performs a specific stage of the workflow and produces an output file that becomes the input of the following stage. This modular structure reduces code dependency, simplifies testing, and allows individual processing stages to be repeated without executing the complete pipeline again.

To verify the modular Artificial Intelligence environment after the preprocessing and PCA preparation stages, the corresponding Python modules were executed and their generated outputs were inspected. Figure 3.6 shows the project-module organization together with an example of the execution output produced during this processing stage.

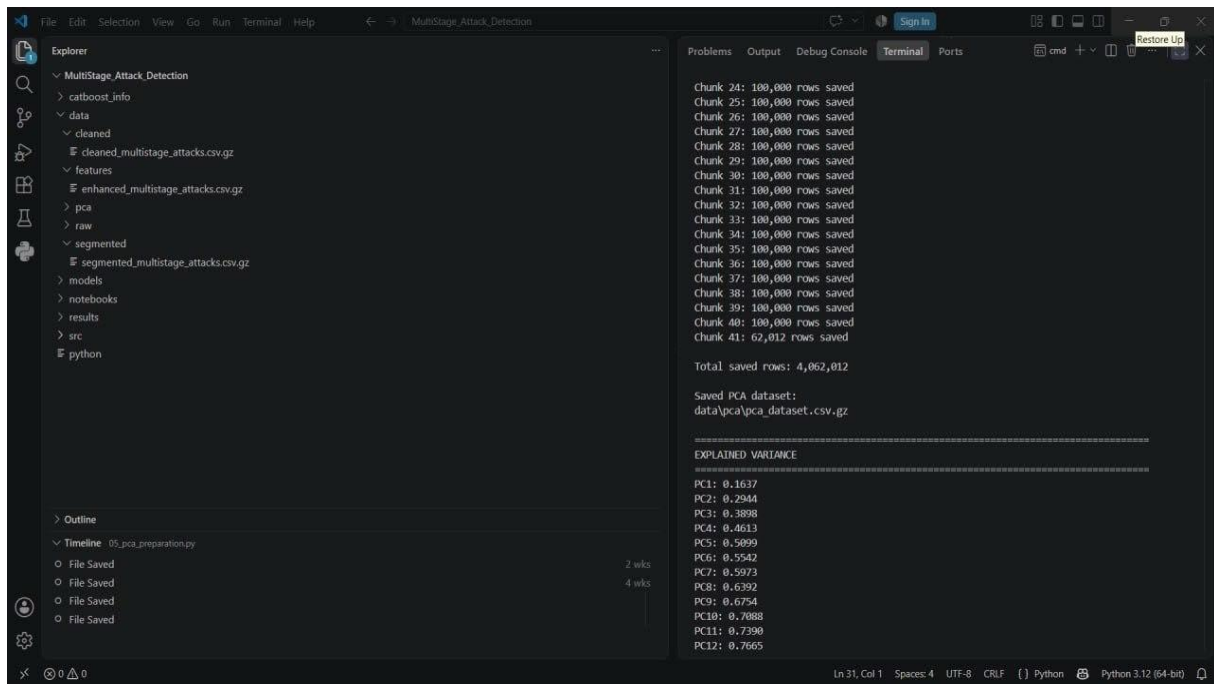


Figure 3.6 Modular Organization and Execution Output of the Artificial Intelligence Project

The main generated datasets are stored in compressed CSV format to reduce storage requirements and support efficient processing. The models, preprocessing objects, evaluation reports, and intermediate outputs are stored in separate project directories to maintain a clear and reproducible implementation structure.

3.8 Dataset Selection and Target Attack Families

The Artificial Intelligence pipeline was developed using network-traffic records derived from the CIC-IDS-2018 dataset. This dataset was selected because it contains benign traffic together with several realistic attack categories and provides a suitable basis for studying complex and multi-stage attack behavior.

The current project focuses on four traffic families: Benign, Bot, Infiltration, and Web Attacks. These families represent different behavioral characteristics and were also included in the adopted attack-pattern segmentation methodology. Benign represents legitimate network activity, whereas Bot, Infiltration, and Web Attacks represent malicious traffic with different levels of complexity and similarity to normal behavior.

Table 3.3 Traffic Families Selected for the Artificial Intelligence Pipeline

Traffic Family	Type	Role in the Project
Benign	Normal	Represents legitimate network behavior
Bot	Malicious	Represents automated bot-related attack behavior
Infiltration	Malicious	Represents malicious activity that may resemble normal traffic
Web Attacks	Malicious	Represents attacks targeting web applications and services

The class-selection module processed the original CIC-IDS-2018 traffic files in chunks and retained only the records belonging to the four target families. Processing the data in chunks reduced memory consumption and allowed the large dataset files to be handled efficiently. The selected records were then combined and stored in a compressed dataset for the subsequent cleaning and feature-engineering stages.

The execution output of the target-class selection stage and the final number of retained records are shown in Figure 3.7.

```
-----
Processing: Thursday-01-03-2018_TrafficForML_CICFlowMeter.csv
-----
Chunk 1: 99,988 selected rows
Chunk 2: 100,000 selected rows
Chunk 3: 100,000 selected rows
Chunk 4: 31,112 selected rows

Selected rows from this file: 331,100

-----
Processing: Wednesday-28-02-2018_TrafficForML_CICFlowMeter.csv
-----
Chunk 1: 99,996 selected rows
Chunk 2: 99,996 selected rows
Chunk 3: 99,995 selected rows
Chunk 4: 99,989 selected rows
Chunk 5: 99,996 selected rows
Chunk 6: 99,996 selected rows
Chunk 7: 13,103 selected rows

Selected rows from this file: 613,071

=====
FINAL SELECTED CLASS COUNTS
=====
Benign: 3,640,843
Bot: 286,191
Web Attacks: 928
Infiltration: 161,934
-----
Total selected rows: 4,089,896
Saved output:
E:\MultiStage_Attack_Detection\data\raw\selected_multistage_attacks.csv.gz
Compressed output size: 357.00 MB

Target class selection completed successfully.
```

Figure 3.7 Target Traffic-Family Selection and Dataset Generation Output

The target-class selection stage retained a total of 4,089,896 traffic records. The selected data consisted of 3,640,843 Benign records, 286,191 Bot records, 161,934 Infiltration records, and 928 Web Attacks records. The resulting dataset was saved as `selected_multistage_attacks.csv.gz` and used as the input to the data-cleaning stage.

After the selection stage, the retained records passed through cleaning, feature engineering, dimensionality reduction, and attack-pattern segmentation. The final segmented dataset was stored as `segmented_multistage_attacks.csv.gz` and contained the principal-component features, the original traffic-family label, the assigned cluster identifier, and the generated subpattern label. This dataset was later used as the direct input to the stacking and hierarchical classification models.

3.9 AI Data-Processing Pipeline

The Artificial Intelligence implementation follows a sequential data-processing pipeline in which the output of each stage becomes the input of the next stage. The pipeline begins with the selection of the target traffic families from the CIC-IDS-2018 dataset and continues

through data cleaning, feature engineering, standardization, dimensionality reduction, attack-pattern segmentation, model training, and final prediction generation.

The pipeline was designed as a modular sequence of Python scripts. Each script performs one clearly defined operation and stores its output in a dedicated project directory. This organization allows individual stages to be tested, repeated, or modified without rerunning the complete workflow from the beginning.

The first stage selects the four target traffic families and combines the retained records into a compressed dataset. The cleaning stage then standardizes column names and labels, removes invalid or missing values, and prepares the numeric traffic features for further processing. A port-variance feature is subsequently generated to provide additional information about communication behavior before the data is standardized and transformed using Principal Component Analysis.

The reduced feature representation is then divided into behavioral subpatterns using class-specific clustering. The segmented dataset is used to train the stacking ensemble and the hierarchical CatBoost models. During prediction, the system first identifies the traffic family and then predicts the corresponding subpattern within that family. The resulting prediction includes the family label, subpattern label, and confidence values required by the subsequent integration stage.

The following sections describe each stage of the Artificial Intelligence pipeline in greater detail, beginning with data cleaning and label preparation.

3.10 Data Cleaning and Label Preparation

The selected CIC-IDS-2018 records were cleaned before feature engineering and model development. This stage was necessary because the original traffic files could contain inconsistent column names, label variations, missing values, infinite values, and non-numeric entries that could negatively affect the subsequent processing stages.

The cleaning module first standardized the dataset structure by removing unnecessary spaces from column names and traffic labels. The retained labels were then normalized so that records belonging to the same traffic family used a consistent class name. In particular, the required web-attack variants were consolidated under the Web Attacks family.

The traffic-feature columns were converted into numeric values. Entries that could not be converted were treated as missing values, while positive and negative infinity values were replaced with undefined values. Rows containing invalid values in the required feature or label columns were then removed to ensure that only valid records continued to the feature-engineering stage.

After cleaning and validation were completed, the number of retained and removed records was reviewed to confirm the effect of the preprocessing stage. The execution summary is shown in Figure 3.8.

```
=====
FINAL CLEANING SUMMARY
=====
Total input rows: 4,089,896
Total clean rows: 4,062,012
Total rows removed: 27,884

Removed rows by reason:
Invalid labels: 0
Missing required values: 0
Invalid numeric/Infinity: 24,456
Duplicates within chunks: 3,428

Final class counts:
Benign: 3,615,320
Bot: 285,157
Web Attacks: 928
Infiltration: 160,607

Saved cleaned dataset:
E:\MultiStage_Attack_Detection\data\cleaned\cleaned_multistage_attacks.csv.gz
Compressed output size: 358.15 MB

Cleaning summary saved:
E:\MultiStage_Attack_Detection\results\cleaning_summary.csv

Dataset cleaning completed successfully.

E:\MultiStage_Attack_Detection\src>
```

Figure 3.8 Final Data-Cleaning Summary and Cleaned-Dataset Generation Output

The cleaning stage processed 4,089,896 input records and retained 4,062,012 valid records. A total of 27,884 records were removed, including 24,456 records containing invalid numeric or infinite values and 3,428 duplicate records. No records were removed because of invalid labels or missing required values. The cleaned dataset was saved as

cleaned_multistage_attacks.csv.gz, while a separate cleaning summary was stored in cleaning_summary.csv.

The cleaned dataset became the input to the port-variance feature-engineering stage.

3.11 Port-Variance Feature Engineering

After the data-cleaning stage, an additional traffic feature was generated to improve the representation of communication behavior. The selected feature was based on the variance of port information, which can help describe changes in service usage and communication patterns across different types of network traffic.

Port information is relevant because different applications, services, and attack tools may produce distinct communication patterns. Instead of using individual port numbers directly, the implementation generated a statistical representation of port behavior. This approach reduces dependence on specific port values while preserving information about changes in network activity.

The feature-engineering module processed the cleaned dataset in chunks and calculated the required port-variance values before appending them to the existing traffic features. Chunk-based processing was used to reduce memory consumption and support the large number of retained traffic records.

After the port-variance features were generated for all cleaned records, their output statistics and the generated enhanced dataset were verified. Figure 3.9 presents the execution summary of this feature-engineering stage.

```
Features added: Port_Variance_W10, Port_Variance_W100, Port_Variance_W1000

=====
PORT VARIANCE FEATURE SUMMARY
=====
Total input rows: 4,062,012
Total output rows: 4,062,012

Port_Variance_W10
Minimum: 0.000000
Maximum: 1062550083.850000
Mean: 240026034.938609

Port_Variance_W100
Minimum: 0.000000
Maximum: 837851790.017601
Mean: 272730813.007284

Port_Variance_W1000
Minimum: 0.000000
Maximum: 611113127.309156
Mean: 279760831.482248

Saved enhanced dataset:
E:\MultiStage_Attack_Detection\data\features\enhanced_multistage_attacks.csv.gz
Compressed output size: 434.85 MB

Saved feature summary:
E:\MultiStage_Attack_Detection\results\port_variance_summary.csv

Port variance feature engineering completed successfully.

E:\MultiStage_Attack_Detection\src>
```

Figure 3.9 Port-Variance Feature Engineering Summary and Enhanced-Dataset Generation Output

The feature-engineering stage processed all 4,062,012 cleaned records and generated three additional features: Port_Variance_W10, Port_Variance_W100, and Port_Variance_W1000. The enhanced dataset was saved as `enhanced_multistage_attacks.csv.gz`, while the statistical summary of the generated features was stored in `port_variance_summary.csv`.

The enhanced dataset became the input to the standardization and Principal Component Analysis stage.

3.12 Data Standardization and PCA Preparation

Before applying attack-pattern segmentation and classification, the enhanced traffic features were standardized and transformed using Principal Component Analysis. Standardization was required because the original features had different numerical ranges, which could cause features with large values to dominate the subsequent learning and clustering processes.

The `StandardScaler` was fitted incrementally on the enhanced dataset. The feature values were processed in chunks rather than loading the complete dataset into memory. Each valid

chunk contributed to the estimation of the global feature means and standard deviations, after which the same fitted scaler was used to transform the records.

Incremental Principal Component Analysis was then applied to the standardized features. The implementation initially allowed a maximum of 30 principal components and selected the minimum number of components required to preserve at least 95% of the cumulative explained variance. This reduced the dimensionality of the traffic data while retaining most of its meaningful information.

Table 3.4 . Standardization and PCA Configuration

Parameter	Configured Value
Input dataset	enhanced_multistage_attacks.csv.gz
Processing method	Chunk-based incremental processing
Chunk size	100,000 records
Standardization method	StandardScaler
PCA method	Incremental PCA
Maximum tested components	30
Explained-variance threshold	95%
Retained components	PC1–PC26
Output dataset	pca_dataset.csv.gz

The resulting representation retained 26 principal components, identified as PC1 through PC26. The fitted StandardScaler and Incremental PCA objects were saved so that the same transformations could be reused during later prediction and integration stages.

After fitting Incremental PCA, the cumulative explained variance was examined to determine the number of principal components required by the configured 95% criterion. The resulting dimensionality-reduction summary is shown in Figure 3.10.

```
Total saved rows: 4,062,012
Saved PCA dataset:
data\pca\pca_dataset.csv.gz

=====
EXPLAINED VARIANCE
=====
PC1: 0.1637
PC2: 0.2944
PC3: 0.3898
PC4: 0.4613
PC5: 0.5099
PC6: 0.5542
PC7: 0.5973
PC8: 0.6392
PC9: 0.6754
PC10: 0.7088
PC11: 0.7390
PC12: 0.7665
PC13: 0.7909
PC14: 0.8082
PC15: 0.8249
PC16: 0.8409
PC17: 0.8548
PC18: 0.8681
PC19: 0.8809
PC20: 0.8934
PC21: 0.9046
PC22: 0.9156
PC23: 0.9257
PC24: 0.9348
PC25: 0.9435
PC26: 0.9510

Reached 95% variance using 26 principal components.
E:\MultiStage_Attack_Detection>
```

Figure 3.10 Explained-Variance Summary and PCA Dataset Generation Output

The PCA stage processed and saved 4,062,012 traffic records. The cumulative explained variance reached 95.10% using 26 principal components. The transformed dataset was saved as `pca_dataset.csv.gz` and became the input to the attack-pattern segmentation stage.

3.13 Attack Pattern Segmentation

After dimensionality reduction, the PCA-transformed traffic records were divided into behavioral subpatterns. The purpose of this stage was to represent the internal behavioral

diversity of each traffic family rather than treating all records belonging to the same family as a single homogeneous class.

Attack-pattern segmentation was performed independently for the four traffic families: Benign, Bot, Infiltration, and Web Attacks. A separate clustering model was therefore used for each family so that its records could be grouped according to their own behavioral characteristics in the PCA feature space.

K-Means-based clustering was applied to the principal-component features, and each resulting cluster was assigned a descriptive subpattern label. The generated labels followed the format Family_Pattern_n, allowing every record to retain both its original traffic-family identity and its more specific behavioral pattern.

Table 3.5 Generated Behavioral Subpatterns

Traffic Family	Number of Subpatterns	Generated Subpatterns	Total Records
Benign	2	Benign_Pattern_1 – Benign_Pattern_2	3,615,320
Bot	5	Bot_Pattern_1 – Bot_Pattern_5	285,157
Infiltration	9	Infiltration_Pattern_1 Infiltration_Pattern_9	160,607
Web Attacks	6	Web Attacks_Pattern_1 – Web Attacks_Pattern_6	928

Total	22	—	4,062,012
-------	----	---	-----------

The segmentation stage assigned a subpattern label and a cluster identifier to every processed traffic record. The resulting dataset preserved the 26 principal-component features together with the original traffic-family label, the assigned Cluster_ID, and the generated Subpattern label.

After class-specific clustering was completed, the generated cluster assignments were reviewed to verify the number and distribution of behavioral subpatterns within each traffic family. Figure 3.11 presents the final segmentation output.

```

=====
Total segmented rows: 4,062,012

Saved segmented dataset:
data\segmented\segmented_multistage_attacks.csv.gz

Saved distribution summary:
results\subpattern_distribution.csv

Final sub-pattern distribution:
Label      Subpattern  Count
Benign     Benign_Pattern_1  2233534
Benign     Benign_Pattern_2  1381786
Bot        Bot_Pattern_1     16656
Bot        Bot_Pattern_2     138100
Bot        Bot_Pattern_3     4911
Bot        Bot_Pattern_4     120507
Bot        Bot_Pattern_5     4983
Infiltration Infiltration_Pattern_1  24015
Infiltration Infiltration_Pattern_2  19085
Infiltration Infiltration_Pattern_3  51228
Infiltration Infiltration_Pattern_4  1224
Infiltration Infiltration_Pattern_5  8575
Infiltration Infiltration_Pattern_6  6086
Infiltration Infiltration_Pattern_7  12808
Infiltration Infiltration_Pattern_8  29440
Infiltration Infiltration_Pattern_9  8146
Web Attacks Web_Attacks_Pattern_1  149
Web Attacks Web_Attacks_Pattern_2  133
Web Attacks Web_Attacks_Pattern_3  157
Web Attacks Web_Attacks_Pattern_4  120
Web Attacks Web_Attacks_Pattern_5  105
Web Attacks Web_Attacks_Pattern_6  264

=====
ATTACK PATTERN SEGMENTATION COMPLETED SUCCESSFULLY
=====

E:\Multistage_Attack_Detection>

```

Figure 3.11 Final Attack-Pattern Segmentation and Subpattern Distribution

The segmentation stage successfully processed all 4,062,012 records and generated a total of 22 behavioral subpatterns. Benign traffic was divided into 2 subpatterns, Bot traffic into 5 subpatterns, Infiltration traffic into 9 subpatterns, and Web Attacks into 6 subpatterns. The final segmented dataset was saved as segmented_multistage_attacks.csv.gz, while the detailed subpattern distribution was stored in subpattern_distribution.csv.

The segmented dataset became the direct input to the machine-learning classification stage, where the generated subpatterns were used as the prediction targets for the stacking and hierarchical classification models.

3.14 Stacking Ensemble Implementation

Following attack-pattern segmentation, a stacking ensemble architecture was adopted to classify the generated behavioral subpatterns. Instead of relying on a single machine-learning algorithm, the framework combines several classifiers to benefit from their complementary learning characteristics.

The stacking architecture combines Random Forest, k-Nearest Neighbors, Logistic Regression, Linear Support Vector Machine, and CatBoost. Each classifier represents a different learning strategy. Random Forest and CatBoost capture complex nonlinear relationships, k-Nearest Neighbors identifies local similarities between behavioral patterns, while Logistic Regression and Linear SVM provide linear and margin-based classification perspectives.

Each base classifier independently generated predictions for the segmented traffic records. The outputs of these classifiers were then combined by a stacking meta-learner to produce the final subpattern prediction. This approach allows the framework to exploit the complementary strengths of multiple classification techniques instead of depending on the behavior of a single model.

The trained stacking architecture was incorporated into the overall Artificial Intelligence detection framework and provided an additional classification mechanism for identifying detailed behavioral subpatterns.

3.15 Hierarchical CatBoost Implementation

An improved hierarchical CatBoost classification architecture was implemented for the segmented traffic data. The purpose of this architecture was to divide the prediction process into two consecutive levels: traffic-family classification followed by detailed subpattern

classification.

The model used the segmented dataset generated in the previous stage. The principal components PC1 through PC26 were used as the input features, while the generated Subpattern field was used to derive both the parent traffic family and the detailed behavioral class.

At the first level, a CatBoost classifier was trained to distinguish among the four traffic families: Benign, Bot, Infiltration, and Web Attacks. After the family was identified, the second level used a dedicated CatBoost classifier associated with that family to predict the corresponding behavioral subpattern. Therefore, the final output contains both a general traffic-family prediction and a more specific subpattern prediction.

To reduce computational requirements while preserving the distribution of the available classes, a stratified sample of up to 400,000 records was used during model development. A fixed random state of 42 was applied for reproducibility, while subpatterns containing fewer than 10 records were excluded to reduce instability during training.

A family-confidence threshold of 0.90 was incorporated into the hierarchical prediction process. This threshold allows the framework to identify cases in which the first-level family prediction is uncertain and to handle these predictions more carefully during the subsequent classification stage.

After training, the family classifier, family-specific subpattern classifiers, feature definitions, and confidence configuration were stored as a reusable model object for later prediction and integration with the Game Theory decision module.

3.16 Prediction Output and Confidence Generation

After model training, the detection framework generated structured prediction outputs for the evaluated traffic records. The prediction process produced both a parent traffic-family label and a detailed behavioral subpattern label, enabling the system to describe detected activity at two different levels.

In addition to the predicted labels, confidence values were generated to represent the reliability of each classification decision. The family-level confidence represents the certainty associated with the first-stage prediction, while the subpattern-level confidence represents the certainty of the detailed prediction generated within the selected traffic family.

The framework also generates a joint-confidence indicator that summarizes the overall reliability of the hierarchical prediction. This information is important because the subsequent defense mechanism requires more than a simple malicious-or-benign decision. A prediction associated with high confidence can be treated differently from a prediction whose classification remains uncertain.

The final prediction information therefore contains the detected traffic family, the corresponding behavioral subpattern, and the associated confidence indicators. These values form the structured output that is transferred to the subsequent integration stage.

This design allows the Artificial Intelligence subsystem to provide detailed and confidence-aware detection information rather than producing only a binary attack decision.

3.17 AI-to-Game-Theory Integration

The Artificial Intelligence subsystem was integrated with the Game Theory decision module to enable adaptive security responses based on the detected traffic behavior. Instead of using the classification output only for reporting purposes, the prediction result was transformed into structured decision-support information that could be used by the defense layer.

The integration stage transfers the predicted traffic family, the detected behavioral subpattern, and the corresponding confidence indicators. It also communicates whether an attack was detected, allowing the Game Theory module to evaluate both the type of suspicious activity and the reliability of the Artificial Intelligence decision.

The exchanged information is organized in a structured format to maintain a consistent interface between the machine-learning environment and the SDN-based security platform. This structure simplifies the transfer of prediction information between independently

implemented components of the system.

This architecture separates detection from decision making. The Artificial Intelligence subsystem identifies and characterizes the observed network behavior, while the Game Theory subsystem uses the received information to determine an appropriate defensive action.

3.18 Dynamic Game Theory Decision Module

Following the Artificial Intelligence prediction stage, the generated detection information was forwarded to the Dynamic Game Theory module. This component is responsible for selecting an appropriate defensive strategy according to the detected attack behavior and the confidence associated with the prediction.

Unlike traditional rule-based defense mechanisms, the proposed approach does not apply the same response to every detected attack. Instead, the Game Theory model evaluates the current security situation and selects the response that provides the most suitable balance between attack mitigation and network availability.

The Game Theory module evaluates the Artificial Intelligence output together with the available network evidence, including traffic intensity, Snort alerts, spoofing information, packet loss, RTT, active-source activity, and the current and previous attack stages. The predicted traffic family, behavioral subpattern, and confidence values therefore provide additional decision evidence rather than serving as the sole basis for selecting the defensive strategy.

The selected defensive action is generated after evaluating the available response alternatives. Depending on the estimated attack severity and the confidence level, the module can recommend maintaining normal communication, applying traffic restrictions, or activating stronger mitigation policies.

This adaptive decision mechanism enables the proposed framework to respond dynamically

by considering both the detected network behavior and the reliability of the Artificial Intelligence prediction before selecting the final defensive action.

3.19 SDN-Based Response Generation

After the Game Theory module determines the most appropriate defensive strategy, the selected decision is transferred to the Software-Defined Networking environment. The SDN controller is responsible for translating the selected strategy into network-level actions that can be enforced automatically.

The proposed framework separates attack detection from traffic control. While the Artificial Intelligence subsystem identifies and characterizes suspicious behavior and the Game Theory module determines the defensive strategy, the SDN environment performs the actual network enforcement according to the selected response.

Depending on the selected strategy, the network may maintain normal communication, restrict suspicious traffic, or block malicious flows. This separation improves the flexibility of the framework because defensive policies can be adapted without modifying the Artificial Intelligence detection models.

The selected response can be translated into OpenFlow rules and applied within the SDN infrastructure, allowing the network to react dynamically according to the detected attack behavior and the defensive strategy selected by the Game Theory module.

3.20 Chapter Summary

This chapter presented the implementation methodology of the proposed framework, including data preparation, feature engineering, PCA, attack-pattern segmentation, machine-learning classification, and AI integration with Game Theory and SDN. These implementation stages provide the foundation for the integrated proposed solution described in Chapter Four and the experimental evaluation presented in Chapter Five.

4 Chapter Four: Proposed Solution

4.1 Introduction

This chapter presents the proposed security framework and describes the main enhancements introduced to the original Software-Defined Networking and Game Theory-based defense architecture. The previously implemented SDN components are retained as the underlying security platform, while the chapter places greater emphasis on the extensions developed during the current work.

The first enhancement focuses on multi-stage attack analysis by incorporating temporal security-state tracking and Markov-based transition modeling. This extension enables the framework to represent the progression of suspicious behavior across successive observation and decision rounds rather than treating each event independently.

The second enhancement introduces an Artificial Intelligence-based detection mechanism capable of identifying the attack family, the corresponding behavioral subpattern, and the associated prediction confidence. The generated AI evidence is subsequently integrated with the multi-stage context and the existing Game Theory decision process to support adaptive mitigation and OpenFlow-based enforcement.

The following sections describe the proposed solution architecture, the multi-stage attack representation, the AI-based threat representation, and the final integration of these components within the adaptive defense process.

4.2 Overall Integrated Solution Architecture

The proposed solution is structured as an integrated security framework that extends the original SDN-based defense environment through two complementary enhancements. The existing platform, consisting of the Ryu controller, OpenFlow, Open vSwitch, Snort IDS, and the Game Theory decision engine, continues to provide the monitoring, decision, and mitigation infrastructure.

The first extension introduces multi-stage attack tracking. Network observations are

evaluated across successive decision rounds, and the previous and current operational states are maintained to capture changes in attack behavior. Markov-based transition information is used to provide temporal context regarding the progression of suspicious activity.

The second extension introduces an Artificial Intelligence-based detection layer. The AI subsystem performs detailed behavioral analysis and produces the predicted attack family, behavioral subpattern, and associated confidence information. These outputs provide additional evidence that complements the previously implemented multi-stage tracking mechanism.

The multi-stage context and AI detection evidence are subsequently incorporated into the adaptive Game Theory decision process. The selected defensive strategy is finally translated into an SDN action and enforced through OpenFlow-based rate limiting or traffic blocking within the Open vSwitch data plane.

4.3 Multi-Stage Attack Tracking and Representation

The first major enhancement introduced to the original defense framework is the incorporation of multi-stage attack tracking. This mechanism was designed to represent the temporal evolution of suspicious network behavior across consecutive observation and decision rounds rather than evaluating each traffic event independently.

The implemented framework maintains state information for each monitored destination and distinguishes between the previously observed security state and the current operational state. This temporal representation enables the controller to identify meaningful changes in network behavior and to determine whether a sequence of observations may indicate the progression of a multi-stage attack.

The multi-stage analysis is based on operational evidence collected from the SDN environment, including traffic intensity, spoofing indicators, Snort alerts, and active-source information. These observations are used to derive the current security state and are subsequently evaluated together with the previously recorded state.

The resulting stage information provides temporal context for the subsequent Markov-based state evaluation and Game Theory decision process. In this manner, the framework extends the original traffic-oriented defense mechanism with an explicit representation of attack progression over time.

4.3.1 Operational Security States

The implemented multi-stage tracking mechanism represents the observed network condition using six operational security states: NORMAL, PROBE, FLOOD_LOW, FLOOD_HIGH, IP_SPOOFING, and SPOOFED_FLOOD. These states provide a structured representation of traffic behavior and allow the framework to distinguish between normal operation, early suspicious activity, flooding behavior of different intensities, spoofing activity, and combined spoofing–flooding conditions.

The current state is determined from multiple observable indicators, including packet rate, Snort alert status, spoofing information, and the number of active traffic sources. This multi-indicator representation is used instead of relying on packet rate alone, thereby providing a more informative description of the current security condition.

The resulting operational state is retained as part of the temporal attack context and is compared with the previously recorded state during subsequent decision rounds. This enables the framework to represent the progression of suspicious behavior and provides the basis for multi-stage transition detection.

Table 4.1 Operational Security States Used in the Multi-Stage Tracking Mechanism

Operational State	Interpretation
NORMAL	Normal traffic without significant attack indicators
PROBE	Early or low-intensity suspicious activity

FLOOD_LOW	Low-intensity flooding behavior
FLOOD_HIGH	High-intensity or multi-source flooding behavior
IP_SPOOFING	Spoofed source-address behavior
SPOOFED_FLOOD	Combined spoofing and flooding behavior

As summarized in Table 4.1, the operational states represent different levels and forms of observed network behavior. The classification conditions used to assign these states and detect transitions between them are described in the following subsection.

4.3.2 Stage Classification and Multi-Stage Transition Detection

The operational security state is determined by evaluating multiple indicators collected during each monitoring round. The implemented controller considers the observed packet rate, Snort alert status, source-address spoofing, and the number of active traffic sources when assigning the current stage.

Three packet-rate thresholds are used to distinguish different levels of traffic intensity. The probing threshold is set to 5 packets per second (pps), the low-flood threshold to 30 pps, and the high-flood threshold to 120 pps. These thresholds are evaluated together with the remaining security indicators rather than being used as independent decision criteria. These values are implementation-specific thresholds used in the emulated environment to separate traffic-intensity levels; they are evaluated together with Snort, spoofing, and active-source evidence rather than treated as universal standalone limits.

In the absence of spoofing and Snort evidence, traffic below the probing threshold is

classified as NORMAL, while traffic between the probing and low-flood thresholds is classified as PROBE. Traffic below the high-flood threshold after exceeding the lower thresholds is classified as FLOOD_LOW, whereas higher traffic intensity is classified as FLOOD_HIGH.

Additional conditions are applied when stronger attack indicators are observed. Spoofed traffic is classified as IP_SPOOFING, while spoofing combined with sufficient traffic intensity or Snort evidence is classified as SPOOFED_FLOOD. Furthermore, traffic originating from two or more active sources may be classified as FLOOD_HIGH when suspicious traffic intensity or IDS evidence is present.

After the current stage is determined, it is compared with the previously recorded stage for the same monitored destination. A multi-stage transition is identified when the observed behavior changes between flooding-related and spoofing-related states, or when distributed-source activity provides additional evidence of evolving malicious behavior. This mechanism enables the framework to distinguish isolated suspicious observations from attack behavior that develops across multiple stages.

4.3.3 Markov-Based Temporal Modeling

To incorporate temporal information into the multi-stage analysis, the proposed framework employs a Markov-based state-transition mechanism. Rather than evaluating the current operational state in isolation, the model considers the previously observed state and the probability of transition toward each possible subsequent state.

For a current state S_i and a subsequent state S_j , the transition relationship is represented as [22]:

$$P(X_{t+1} = S_j \mid X_t = S_i)$$

where X_t denotes the operational security state at decision round t . The implemented transition model assigns a probability distribution to each possible next state for every operational state.

Table 4.2 Markov Transition Probability Matrix Used in the Multi-Stage Model

Previous State	NORMAL	PROBE	FLOOD – LOW	FLOOD_ HIGH	IP_ SPOOFING	SPOOFED FLOOD
NORMAL	0.80	0.15	0.03	0.01	0.01	0.00
PROBE	0.15	0.45	0.25	0.08	0.05	0.02
FLOOD_LOW	0.05	0.10	0.45	0.25	0.05	0.10
FLOOD_HIGH	0.03	0.05	0.12	0.55	0.05	0.20
IP_SPOOFING	0.03	0.07	0.10	0.10	0.50	0.20
SPOOFED_ FLOOD	0.02	0.03	0.05	0.15	0.10	0.65

As shown in Table 4.2, each row represents the probability distribution of the subsequent operational state given the previously observed state. The relatively high diagonal probabilities preserve temporal continuity, while the off-diagonal probabilities allow the model to represent progression toward different attack conditions.

For example, the NORMAL-to-NORMAL transition can be expressed as $P(X_{t+1}=\text{NORMAL} \mid X_t=\text{NORMAL})=0.80$. This means that when the current operational state is NORMAL, the model assigns an 80% probability of remaining in the

NORMAL state in the next decision round. The remaining entries of the transition matrix are interpreted in the same manner [22].

Based on the Markov transition principle [22], the implemented framework combines the current evidence with the transition prior using the following project-defined weighting:

$$Bt(s) = 0.65 Et(s) + 0.35 Pt(s | St-1)$$

where $Et(s)$ represents the belief derived from the current observed evidence, $Pt(s | St-1)$ represents the Markov transition prior determined by the previous state, and $Bt(s)$ represents the combined belief for state s .

The current observed stage and additional multi-stage indicators are subsequently used to adjust the combined belief distribution. In particular, the current stage receives additional weight, while detected multi-stage behavior increases the belief associated with severe flooding and spoofed-flooding states. The resulting values are normalized to produce the final state-belief distribution used by the Game Theory decision engine.

4.3.4 Multi-Stage Evidence in the Defense Process

The output of the multi-stage analysis is incorporated into the decision-making process as contextual security evidence. The implemented Game Theory engine receives both instantaneous network observations and temporal attack information, enabling the defensive decision to reflect not only the current traffic condition but also the evolution of the observed behavior.

The multi-stage context supplied to the decision engine includes the previous operational state, the current operational state, the multi-stage detection flag, and the number of active traffic sources. These attributes are evaluated together with packet rate, Snort alert status, packet loss, round-trip time, protocol information, and spoofing indicators.

The multi-stage flag directly influences the state-belief estimation process by reducing the belief associated with benign or early suspicious states and increasing the belief associated with more severe flooding-related conditions. Active-source and spoofing information

provide additional adjustments to the final state-belief distribution.

Consequently, multi-stage evidence is not treated as an isolated detection result. Instead, it becomes part of the contextual information used by the Game Theory engine to estimate the current threat condition and subsequently evaluate the available defensive strategies.

4.4 AI-Based Attack Detection and Threat Representation

Following the implementation of the multi-stage attack tracking mechanism, the proposed framework was further enhanced by integrating an Artificial Intelligence-based detection layer. The objective of this enhancement is to provide a more detailed characterization of malicious traffic by identifying both the attack family and the corresponding behavioral subpattern.

Unlike the original framework, which relied primarily on network observations and temporal attack progression, the AI component performs data-driven behavioral analysis using machine learning models trained on segmented attack patterns. This additional layer provides more detailed attack identification while also producing quantitative confidence information for each prediction

The AI detection output is not intended to replace the multi-stage analysis. Instead, it complements the existing temporal attack representation by supplying higher-level semantic information that is subsequently incorporated into the adaptive Game Theory decision process.

4.4.1 Attack Pattern Segmentation

The first stage of the AI pipeline consists of attack pattern segmentation, which partitions network traffic into behaviorally similar groups before classification. Instead of directly training a classifier on the original attack categories, segmentation is performed to expose hidden behavioral differences among attacks belonging to the same family.

The segmentation process is applied after feature preparation and dimensionality reduction

and produces a collection of behavioral subpatterns that provide a finer representation of attack characteristics. These subpatterns subsequently become the target classes used by the hierarchical classification framework.

By separating attack families into multiple behavioral patterns, the framework reduces the behavioral variability represented by each target class and provides more focused subpattern labels for the subsequent classification stage.

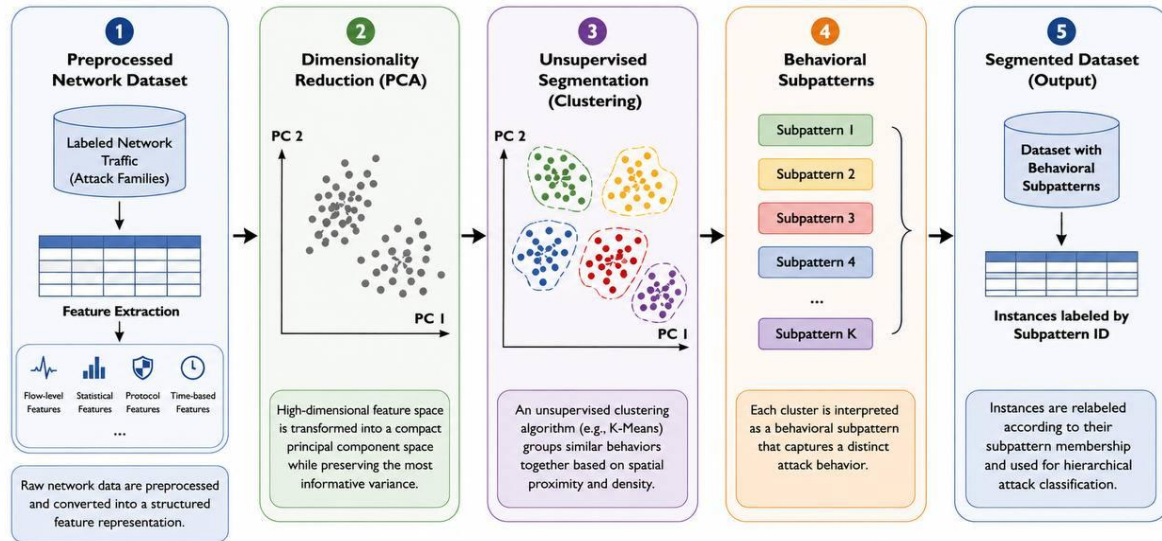


Figure 4.1 Attack Pattern Segmentation Process in the Proposed AI Framework

Figure 4.1 illustrates the attack pattern segmentation process adopted in the proposed AI framework. The process begins with the preprocessed network dataset, followed by dimensionality reduction using Principal Component Analysis (PCA). Unsupervised clustering is then applied to identify groups of traffic instances with similar behavioral characteristics. Each resulting cluster is interpreted as a behavioral subpattern, and the corresponding subpattern labels are assigned to the original instances to produce the segmented dataset used in the subsequent hierarchical classification stage.

4.4.2 Hierarchical Attack Classification

Following attack pattern segmentation, the segmented dataset is used to train a hierarchical attack classification framework. Instead of directly predicting individual attack subpatterns,

the proposed AI pipeline performs the classification process in two successive levels to preserve the family–subpattern relationship and restrict the detailed prediction task to the relevant family.

The first classification level identifies the attack family, including Benign, Bot, Infiltration, and Web Attacks. Once the attack family has been determined, the corresponding family-specific classifier is activated to identify the behavioral subpattern associated with the observed traffic instance.

This hierarchical prediction strategy reduces the complexity of the classification problem by restricting the second-stage prediction to subpatterns belonging only to the predicted attack family. Consequently, the second-stage classifier compares only the subpatterns belonging to the selected family rather than all generated subpatterns simultaneously.

Table 4.3 Hierarchical Classification Levels

Classification Level	Output
Level 1	Attack Family
Level 2	Behavioral Subpattern

Table 4.3 summarizes the two-stage hierarchical classification process adopted by the proposed AI framework. The first level predicts the general attack family, whereas the second level refines the prediction by identifying the corresponding behavioral subpattern within the predicted family.

The hierarchical structure enables each family-specific classifier to focus on a limited subset of behavioral patterns, thereby improving discrimination capability and reducing classification ambiguity among unrelated attack categories.

4.4.3 Confidence-Aware Hierarchical Prediction

To improve the reliability of the hierarchical classification process, the proposed AI framework incorporates confidence-aware prediction at both classification levels. The family classifier first produces a probability distribution over the available attack families, while the selected family-specific classifier produces the corresponding subpattern probabilities.

In the implemented hierarchical framework, a project-defined joint confidence score is calculated to represent the confidence of the complete family–subpattern prediction path. It combines the probability of the predicted attack family with the conditional probability of the selected behavioral subpattern as follows:

$$C_{joint} = P(Family) \times P(Subpattern | Family)$$

where $P(Family)$ represents the probability assigned to the selected attack family and $P(Subpattern | Family)$ represents the probability assigned to the selected behavioral subpattern by the corresponding family-specific classifier.

The framework also applies a family-confidence threshold of 0.90. When the probability of the highest-ranked family falls below this threshold, the prediction is treated as a low-confidence family decision. In this case, the second-ranked family is also evaluated through its corresponding subpattern classifier, and the joint confidence scores of the two candidate prediction paths are compared.

The final hierarchical prediction is therefore selected according to the family–subpattern path that produces the higher joint confidence score. This mechanism reduces the risk of propagating an uncertain family-level prediction directly into the subpattern classification stage.

4.4.4 AI-to-Defense Acceptance Condition

After the hierarchical prediction is generated, an additional validation step is applied before the AI result is incorporated into the defense decision process. This step ensures that only sufficiently reliable malicious predictions are forwarded to the Game Theory component.

The final joint confidence score is used as the primary reliability indicator. In the

implemented AI-to-defense bridge, the value 0.50 is used as a configured minimum forwarding threshold for detected malicious traffic. The resulting project-defined acceptance condition is expressed as:

$$Send_To_Game_Theory = Attack_Detected \wedge (C_{joint} \geq 0.50)$$

where C_{joint} denotes the final joint confidence obtained from the hierarchical prediction process.

For interpretability, the joint confidence is additionally mapped into three confidence levels.

Predictions with a joint confidence greater than or equal to 0.90 are labeled HIGH, values between 0.70 and 0.90 are labeled MEDIUM, and values below 0.70 are labeled LOW.

Table 4.4 Confidence Thresholds Used in the AI Decision Pipeline

Threshold / Level	Value	Purpose
Family Confidence Threshold	0.90	Determines whether the second-ranked family should also be evaluated
HIGH Confidence	≥ 0.90	Indicates a highly confident final hierarchical prediction
MEDIUM Confidence	$0.70 - < 0.90$	Indicates a moderately confident prediction
LOW Confidence	< 0.70	Indicates a low-confidence prediction

Defense Acceptance Threshold	≥ 0.50	Determines whether a detected malicious prediction is forwarded to Game Theory
------------------------------	-------------	--

The thresholds in Table 4.4 serve different implementation purposes and are not treated as interchangeable or universally optimal values. The 0.90 family-confidence threshold is used conservatively to determine whether the highest-ranked family can be used directly or whether the second-ranked family should also be evaluated. The 0.70 boundary is used only to describe the final prediction as HIGH, MEDIUM, or LOW confidence and does not determine whether the result is forwarded. The 0.50 defense-acceptance threshold acts as the minimum gate for a detected malicious prediction: results below this value are not forwarded, while accepted results are still treated as additional evidence and are combined with the existing network and multi-stage observations by the Game Theory engine.

4.5 Integrated Multi-Stage and AI Decision Evidence

After the multi-stage tracking mechanism and the AI-based detection layer have been established, their outputs are integrated to form a unified security context for the adaptive defense process. The objective of this integration is to combine temporal attack progression with detailed AI-based behavioral identification before the Game Theory engine evaluates the available defensive strategies.

The multi-stage component provides temporal and network-level evidence, including the previous operational state, current operational state, multi-stage transition status, spoofing information, and the number of active traffic sources. In parallel, the AI component provides the detected attack family, behavioral subpattern, and joint prediction confidence.

Only AI detections that satisfy the acceptance condition defined in Section 4.4.4 are considered valid for the integrated defense process. The accepted AI information is then used as additional contextual evidence alongside the existing multi-stage observations.

In the implemented controller, the presence of a valid AI detection strengthens the multi-stage attack evidence. Therefore, the AI result does not independently replace the state-based analysis; rather, it reinforces the temporal security context that is subsequently provided to the Game Theory decision engine.

4.6 Dynamic Game-Theoretic Defense Decision

The integrated security evidence is subsequently processed by the dynamic Game Theory engine to determine an appropriate defensive response. Unlike a static decision mechanism, the implemented engine maintains contextual information across consecutive decision rounds and adapts the selected strategy according to the estimated security state, historical behavior, and current network observations.

The decision process begins by estimating a belief distribution over the operational security states using the available evidence. The instantaneous evidence-based beliefs are combined with the Markov transition context described in Section 4.3.3, thereby incorporating both current observations and temporal attack progression.

The resulting state-belief distribution is then used to update a dynamic malicious-reputation value associated with the monitored traffic flow. This reputation provides additional historical context and influences the utility assigned to the available defensive strategies.

Finally, the Game Theory engine calculates the expected utility of each defensive action and selects the strategy that maximizes the defender's utility under the current security conditions. The selected strategy is subsequently translated into an SDN mitigation action.

4.6.1 Evidence-Based Attacker Beliefs

The dynamic Game Theory engine estimates a belief distribution over the operational security states using evidence collected from the monitored traffic. The belief estimation process considers traffic intensity together with additional indicators such as Snort alerts, source-address spoofing, protocol type, multi-stage behavior, and the number of active traffic

sources.

To account for differences in traffic intensity, the observed packet rate is first normalized with respect to a reference traffic rate. In the implemented Game Theory engine, the normalized traffic rate is converted into an attack-strength value using the following project-defined logarithmic transformation:

$$A = \ln(1 + r / r_{base})$$

where r represents the observed packet rate and r_{base} represents the reference packet rate. The resulting attack-strength value and the remaining security indicators are used to calculate a raw score for each possible operational state. These scores are then transformed into a probability distribution using the Softmax function [23]:

$$BE(s) = \exp(qs - q_{max}) / \sum_k \exp(qk - q_{max})$$

where qs denotes the raw evidence score assigned to state s and q_{max} is the maximum state score used to improve numerical stability.

The resulting evidence-based distribution is subsequently combined with the Markov transition prior according to the temporal belief model described previously in Section 4.3.3. Therefore, the final attacker-state belief reflects both instantaneous network evidence and the temporal evolution of the observed attack.

4.6.2 Dynamic Malicious Reputation

To preserve historical information across consecutive decision rounds, the dynamic Game Theory engine maintains a malicious-reputation value for each monitored traffic flow. The reputation is updated according to the estimated probability that the observed flow is malicious, allowing the decision process to account for previous behavior rather than relying exclusively on the current observation.

In the implemented state model, the malicious probability is defined from the final state-belief distribution as:

$$P_{malicious} = 1 - B(NORMAL)$$

where $B(NORMAL)$ represents the estimated probability that the current traffic condition corresponds to the NORMAL state.

The reputation value is then updated using an exponentially weighted formulation [24]:

$$R_t = (1 - \alpha)R_{t-1} + \alpha P_{malicious}$$

where R_{t-1} is the previously stored reputation, $P_{malicious}$ is the current malicious probability, and α controls the contribution of the current observation. In the implemented Game Theory engine, α is set to 0.30.

To reduce the stored reputation when stronger evidence of normal traffic is observed, the implemented engine applies the following project-defined cooling term:

$$R_t = R_t - 0.25B(NORMAL)$$

The resulting reputation value is finally constrained to the interval [0,1]. This mechanism enables malicious behavior to increase the historical risk associated with a flow while allowing the reputation to gradually decrease when subsequent observations provide stronger evidence of normal traffic.

4.6.3 Defender Payoff Model

The Game Theory engine evaluates five defensive actions: ALLOW, RL_1, RL_2, RL_3, and BLOCK. Each action is assigned a predefined payoff for every operational security state. The payoff values represent the relative benefit or cost of applying a specific defensive response under a given network condition.

Table 4.5 presents the defender payoff matrix implemented in the current Game Theory engine.

State	ALLOW	RL_1	RL_2	RL_3	BLOCK
NORMAL	12	3	0	-2	-15
PROBE	4	7	5	2	-3
FLOOD_LOW	-4	7	8.5	5	4
FLOOD_HIGH	-12	1	7	9	12
IP_SPOOFING	-6	4	8	6	5
SPOOFED_FLOOD	-15	0	6	9	14

The matrix values are implementation-defined relative utilities: higher values favor an action under the corresponding state, whereas negative values penalize responses that are undesirable for that condition.

4.6.4 Expected Utility Calculation

For each available defensive action, the Game Theory engine calculates an expected utility using the current belief distribution and the predefined defender payoff matrix. The base expected utility of an action a is calculated according to the expected-utility principle used in cybersecurity game modeling [9]:

$$U_{base}(a) = \sum_s B(s) \times M(s,a)$$

where $B(s)$ represents the current belief associated with security state s , and $M(s,a)$ represents the predefined payoff of applying defensive action a under that state.

The calculated utility is subsequently adjusted according to the current malicious reputation and contextual attack evidence. These dynamic adjustments allow the same predefined payoff matrix to produce different utility values across successive decision rounds.

4.6.5 Optimal Defense Selection

After calculating the adjusted expected utilities of all available defensive actions, the Game Theory engine selects the action with the highest utility as the optimal defensive response.

The action with the highest utility is selected as the optimal defensive response, consistent with game-theoretic cyber-defense decision making [9], [25]:

$$d^* = \arg \max U(d)$$

where $U(d)$ represents the expected utility of defensive action d .

The selected action is then mapped to its corresponding rate-limiting value and forwarded to the SDN mitigation layer for enforcement.

4.7 Adaptive Mitigation Strategy

The defensive strategy selected by the Game Theory engine is translated into an adaptive mitigation action within the SDN controller. Instead of applying a single static response to all attacks, the proposed framework dynamically adjusts the mitigation level according to the estimated attack severity.

Five mitigation strategies are supported by the implemented controller: ALLOW, RL_1,

RL_2, RL_3, and BLOCK. These strategies provide progressive protection levels ranging from normal forwarding to complete traffic blocking.

Table 4.6 Adaptive Mitigation Actions

Strategy	Action
ALLOW	Normal forwarding
RL_1	Light rate limiting
RL_2	Moderate rate limiting
RL_3	Aggressive rate limiting
BLOCK	Drop all traffic

Table 4.6 summarizes the mitigation actions implemented by the SDN controller. The action selected by the Game Theory engine is translated into the corresponding OpenFlow rule or rate-limiting configuration for network enforcement.

4.8 Chapter Summary

This chapter presented the implementation of the proposed adaptive SDN security framework. The framework integrates multi-stage attack tracking, Artificial Intelligence-based attack detection, and a dynamic Game Theory decision engine to improve attack

awareness and support adaptive mitigation. The next chapter presents the experimental evaluation and performance analysis of the proposed framework.

5 Chapter Five: Experimental Results and Evaluation

5.1 Introduction

This chapter presents the experimental evaluation of the enhanced security framework. The evaluation focuses on the results obtained from multi-stage attack tracking, Artificial Intelligence-based detection, hierarchical classification, and the integration of these components with the Game Theory and SDN mitigation mechanisms.

The results are presented according to the development sequence of the framework. Multi-stage attack behavior is evaluated first, followed by the AI detection results and the final integrated defense decisions.

5.2 Multi-Stage Attack Evaluation Results

The multi-stage attack mechanism was evaluated by observing the progression of malicious behavior across consecutive monitoring and decision rounds. The framework successfully maintained the previous and current operational states and identified changes in attack behavior over time.

The evaluation began with an ICMP-based attack scenario generated from the Mininet hosts toward the designated victim. The objective was to verify whether the monitoring components could first observe the malicious traffic and then track changes in its behavior across successive controller rounds.

```
mininet@mininet-virtual-machine: ~  
len=28 ip=10.0.0.4 ttl=64 id=14947 icmp_seq=37 rtt=26.3 ms  
len=28 ip=10.0.0.4 ttl=64 id=14975 icmp_seq=38 rtt=24.2 ms  
len=28 ip=10.0.0.4 ttl=64 id=15013 icmp_seq=39 rtt=29.6 ms  
len=28 ip=10.0.0.4 ttl=64 id=15099 icmp_seq=40 rtt=65.4 ms  
len=28 ip=10.0.0.4 ttl=64 id=15101 icmp_seq=41 rtt=16.7 ms  
len=28 ip=10.0.0.4 ttl=64 id=15120 icmp_seq=42 rtt=24.0 ms  
len=28 ip=10.0.0.4 ttl=64 id=15168 icmp_seq=43 rtt=34.0 ms  
len=28 ip=10.0.0.4 ttl=64 id=15204 icmp_seq=44 rtt=16.1 ms  
len=28 ip=10.0.0.4 ttl=64 id=15225 icmp_seq=45 rtt=45.1 ms  
len=28 ip=10.0.0.4 ttl=64 id=15237 icmp_seq=46 rtt=30.7 ms  
len=28 ip=10.0.0.4 ttl=64 id=15273 icmp_seq=47 rtt=38.2 ms  
len=28 ip=10.0.0.4 ttl=64 id=15296 icmp_seq=48 rtt=11.3 ms  
len=28 ip=10.0.0.4 ttl=64 id=15326 icmp_seq=49 rtt=23.5 ms  
  
--- 10.0.0.4 hping statistic ---  
50 packets transmitted, 50 packets received, 0% packet loss  
round-trip min/avg/max = 5.4/29.6/82.0 ms  
mininet> h1 hping3 --icmp -c 500 -i u1000 10.0.0.4
```

Figure 5.1 Multi-Stage Attack Traffic Generation in Mininet

Figure 5.1 confirms that the ICMP attack traffic was generated successfully toward the victim host. This traffic formed the input to the subsequent Snort detection and controller-side multi-stage analysis.

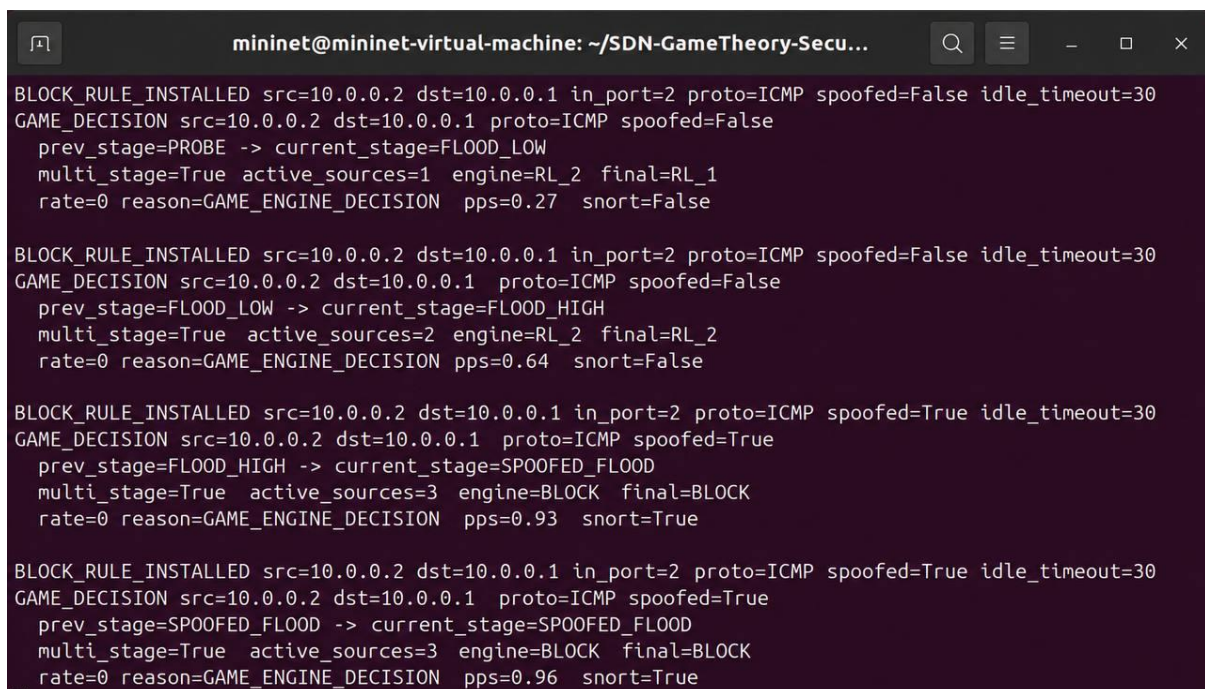
The generated traffic was subsequently monitored by the security components of the framework. Snort IDS detected suspicious network activity, while the controller analyzed the observed behavior across consecutive rounds.

```
user@ubuntu: ~  
[SENT] 2026-06-29 23:25:31 PROJECT_ICMP_ATTACK src=10.0.0.1 dst=10.0.0.4 source_  
log=snort_s1.log  
[SENT] 2026-06-29 23:25:31 PROJECT_ICMP_ATTACK src=10.0.0.4 dst=10.0.0.1 source_  
log=snort_s1.log  
[SENT] 2026-06-29 23:25:45 PROJECT_ICMP_ATTACK src=10.0.0.99 dst=10.0.0.4 source_  
log=snort_s1.log  
[SENT] 2026-06-29 23:25:47 PROJECT_ICMP_ATTACK src=10.0.0.4 dst=10.0.0.1 source_  
log=snort_s2.log  
[SENT] 2026-06-29 23:25:49 PROJECT_ICMP_ATTACK src=10.0.0.4 dst=10.0.0.1 source_  
log=snort_s2.log  
[SENT] 2026-06-29 23:25:49 PROJECT_ICMP_ATTACK src=10.0.0.99 dst=10.0.0.4 source_  
log=snort_s2.log  
[SENT] 2026-06-29 23:25:51 PROJECT_ICMP_ATTACK src=10.0.0.99 dst=10.0.0.4 source_  
log=snort_s1.log  
[SENT] 2026-06-29 23:25:52 PROJECT_ICMP_ATTACK src=10.0.0.99 dst=10.0.0.4 source_  
log=snort_s2.log  
[SENT] 2026-06-29 23:26:13 PROJECT_ICMP_ATTACK src=10.0.0.99 dst=10.0.0.4 source_  
log=snort_s1.log
```

Figure 5.2 Snort Detection of Malicious Traffic during the Multi-Stage Scenario

Figure 5.2 confirms that the generated malicious traffic was detected by Snort IDS during the experiment. The generated alert therefore provided IDS evidence that could be combined with the traffic observations collected by the Ryu controller in the following decision rounds.

After the attack traffic and Snort evidence were observed, the controller evaluated consecutive monitoring rounds to determine whether the traffic represented an isolated event or an evolving multi-stage sequence. The expected behavior was a transition toward more severe operational states as flooding, spoofing, and multiple-source evidence accumulated.



```
mininet@mininet-virtual-machine: ~/SDN-GameTheory-Secu...
BLOCK_RULE_INSTALLED src=10.0.0.2 dst=10.0.0.1 in_port=2 proto=ICMP spoofed=False idle_timeout=30
GAME_DECISION src=10.0.0.2 dst=10.0.0.1 proto=ICMP spoofed=False
  prev_stage=PROBE -> current_stage=FLOOD_LOW
  multi_stage=True active_sources=1 engine=RL_2 final=RL_1
  rate=0 reason=GAME_ENGINE_DECISION pps=0.27 snort=False

BLOCK_RULE_INSTALLED src=10.0.0.2 dst=10.0.0.1 in_port=2 proto=ICMP spoofed=False idle_timeout=30
GAME_DECISION src=10.0.0.2 dst=10.0.0.1 proto=ICMP spoofed=False
  prev_stage=FLOOD_LOW -> current_stage=FLOOD_HIGH
  multi_stage=True active_sources=2 engine=RL_2 final=RL_2
  rate=0 reason=GAME_ENGINE_DECISION pps=0.64 snort=False

BLOCK_RULE_INSTALLED src=10.0.0.2 dst=10.0.0.1 in_port=2 proto=ICMP spoofed=True idle_timeout=30
GAME_DECISION src=10.0.0.2 dst=10.0.0.1 proto=ICMP spoofed=True
  prev_stage=FLOOD_HIGH -> current_stage=SPOOFED_FLOOD
  multi_stage=True active_sources=3 engine=BLOCK final=BLOCK
  rate=0 reason=GAME_ENGINE_DECISION pps=0.93 snort=True

BLOCK_RULE_INSTALLED src=10.0.0.2 dst=10.0.0.1 in_port=2 proto=ICMP spoofed=True idle_timeout=30
GAME_DECISION src=10.0.0.2 dst=10.0.0.1 proto=ICMP spoofed=True
  prev_stage=SPOOFED_FLOOD -> current_stage=SPOOFED_FLOOD
  multi_stage=True active_sources=3 engine=BLOCK final=BLOCK
  rate=0 reason=GAME_ENGINE_DECISION pps=0.96 snort=True
```

Figure 5.3 . Multi-Stage Attack State Transition Detected by the Ryu Controller

Figure 5.3 shows that the controller did not treat the observations as one fixed attack condition. The recorded behavior progressed from less severe suspicious/flooding states toward FLOOD_HIGH and SPOOFED_FLOOD conditions. As the evidence became stronger, the selected response also became more restrictive, demonstrating the role of temporal attack progression in the defense process.

The resulting multi-stage evidence was then provided to the Game Theory decision process, where the estimated attack condition was used to evaluate the available defensive strategies and select an appropriate mitigation response.


```
mininet@mininet-virtual-machine: ~  
5,0.0839,0.0418,0.0297,0.016,0.0131,9.3049,3.12,0.8894,-1.0728,-14.4967,17,GAME_  
ENGINE_DECISION  
2026-06-29 23:33:44,10.0.0.4,10.0.0.1,ICMP,1,0.0,0,0,ALLOW,ALLOW,10000,0.0,0.815  
5,0.0839,0.0418,0.0297,0.016,0.0131,9.3049,3.12,0.8894,-1.0728,-14.4967,17,GAME_  
ENGINE_DECISION  
2026-06-29 23:33:44,10.0.0.99,10.0.0.4,ICMP,4,202.0,0,1,BLOCK,BLOCK,0,0.3699,0.0  
002,0.0195,0.0497,0.1512,0.0759,0.7035,-13.3846,0.9356,6.6984,8.9486,10.4899,1,G  
AME_ENGINE_DECISION  
2026-06-29 23:33:47,10.0.0.1,10.0.0.4,ICMP,4,0.0,0,0,ALLOW,ALLOW,10000,0.0,0.815  
5,0.0839,0.0418,0.0297,0.016,0.0131,9.3049,3.12,0.8894,-1.0728,-14.4967,18,GAME_  
ENGINE_DECISION  
2026-06-29 23:33:47,10.0.0.4,10.0.0.1,ICMP,1,0.0,0,0,ALLOW,ALLOW,10000,0.0,0.815  
5,0.0839,0.0418,0.0297,0.016,0.0131,9.3049,3.12,0.8894,-1.0728,-14.4967,18,GAME_  
ENGINE_DECISION  
2026-06-29 23:33:47,10.0.0.99,10.0.0.4,ICMP,1,423.0,0,1,BLOCK,BLOCK,0,0.5589,0.0  
,0.0094,0.0373,0.2,0.0303,0.723,-14.5086,0.7952,6.9386,9.6235,11.9707,2,GAME_ENG  
INE_DECISION  
2026-06-29 23:33:50,10.0.0.99,10.0.0.4,ICMP,1,0.0,0,1,RL_3,RL_3,512,0.6437,0.086  
4,0.0657,0.0539,0.0383,0.4155,0.3402,-8.045,3.0116,7.1502,7.2442,5.4545,3,GAME_E  
NGINE_DECISION  
2026-06-29 23:33:53,10.0.0.99,10.0.0.4,ICMP,1,0.0,0,1,BLOCK,BLOCK,0,0.7031,0.086  
4,0.0657,0.0539,0.0383,0.4155,0.3402,-8.1162,3.0591,7.2452,7.6748,7.8326,4,GAME_  
ENGINE_DECISION
```

Figure 5.4 Game-Theoretic Defense Decision Based on Multi-Stage Evidence

Figure 5.4 shows the decisions generated after the Game Theory engine evaluated the observed security context. Different records produced different mitigation levels, including ALLOW, rate-limiting actions, and BLOCK, rather than applying one fixed response to every observation. The selected action therefore reflects the estimated attack condition and the utility of the available defense alternatives.

5.3 AI-Based Hierarchical Classification Results

The improved hierarchical CatBoost model was evaluated using separate validation and test datasets. The evaluation included family accuracy, subpattern accuracy, macro precision, macro recall, macro F1-score, weighted precision, weighted recall, weighted F1-score, and the percentage of low-confidence predictions.

The validation and test outputs were then compared to determine whether the hierarchical model maintained similar performance on unseen data rather than performing strongly only on the validation split.


```

Validation results:
Family Accuracy      : 0.9820
Subpattern Accuracy  : 0.9745
Precision Macro      : 0.9412
Recall Macro        : 0.9368
F1 Macro            : 0.9390
Precision Weighted   : 0.9804
Recall Weighted      : 0.9786
F1 Weighted         : 0.9795
Low-confidence rows  : 4.80%

Test results:
Family Accuracy      : 0.9805
Subpattern Accuracy  : 0.9728
Precision Macro      : 0.9365
Recall Macro        : 0.9324
F1 Macro            : 0.9344
Precision Weighted   : 0.9788
Recall Weighted      : 0.9974
F1 Weighted         : 0.9781
Low-confidence rows  : 5.10%

Final metrics:
Split      Family_Accuracy  Subpattern_Accuracy  Precision_Macro  Recall_Macro  F1_Macro  Precision_Weighted  Recall_Weighted  F1_Weighted  Low_Confidence
-----
Validation  0.982000           0.974500           0.941200        0.936800      0.939000      0.980400          0.978600        0.979500      4.80%
Test       0.980500           0.972800           0.936500        0.932400      0.934400      0.978800          0.997400        0.978100      5.10%
-----

Metrics saved: E:\MultiStage_Attack_Detection\results\hierarchical_improved_metrics.csv
-----
IMPROVED HIERARCHICAL CLASSIFICATION COMPLETED SUCCESSFULLY
-----

E:\MultiStage_Attack_Detection>

```

Figure 5.5 Validation and Test Performance of the Improved Hierarchical CatBoost Model

As shown in Figure 5.5 the model achieved a family accuracy of 98.20% on the validation set and 98.05% on the test set. Subpattern accuracy reached 97.45% for validation and 97.28% for testing. These results show that the hierarchical model maintained high accuracy at both the general family level and the more detailed subpattern level.

The macro-averaged metrics also remained high. Macro precision was 94.12% for validation and 93.65% for testing, while macro recall reached 93.68% and 93.24%, respectively. The corresponding macro F1-scores were 93.90% and 93.44%. These values indicate consistent classification performance across the different subpattern classes.

The weighted metrics also showed strong overall performance. Weighted precision reached 98.04% on the validation set and 97.88% on the test set. Weighted recall was 97.86% for validation and 99.74% for testing, while the weighted F1-score reached 97.95% and 97.81%, respectively.

Low-confidence predictions represented 4.80% of the validation samples and 5.10% of the test samples. This indicates that only a relatively small proportion of the evaluated samples were classified with confidence below the predefined family-confidence threshold.

Overall, the results shown in Figure 5.5 demonstrate that the improved hierarchical CatBoost model achieved high classification performance on both validation and test data. The strong family and subpattern accuracies, together with the high macro and weighted metrics and the limited percentage of low-confidence predictions, support the effectiveness of the hierarchical classification approach used in the proposed framework.

5.4 Weak Subpattern Analysis Results

After evaluating the hierarchical classifier, an additional error-analysis stage was performed to identify the behavioral subpatterns that remained difficult to classify. The analysis examined precision, recall, true positives, false positives, and the classes responsible for the most frequent false-positive errors. Subpatterns with precision below 0.80 were considered weak patterns requiring further analysis.

This analysis was performed to determine whether the high overall classification scores concealed specific subpatterns that were still difficult to distinguish.

```
=====
LOW PRECISION SUBPATTERNS (< 0.80)
=====
```

Subpattern	TP	FP	Precision	Recall	Most_FP_From	Most_FP_From_Count
Web_Attacks_Pattern_5	0	0	0.000000	0.000000	None	0
Web_Attacks_Pattern_2	0	0	0.000000	0.000000	None	0
Infiltration_Pattern_6	1	10	0.090909	0.010417	Benign_Pattern_1	10
Infiltration_Pattern_5	3	5	0.375000	0.022222	Benign_Pattern_1	5
Infiltration_Pattern_9	5	6	0.454545	0.038760	Benign_Pattern_2	5
Infiltration_Pattern_3	34	51	0.400000	0.042131	Benign_Pattern_1	51
Infiltration_Pattern_2	19	31	0.380000	0.063123	Benign_Pattern_1	31
Infiltration_Pattern_1	28	22	0.560000	0.074074	Benign_Pattern_2	21
Infiltration_Pattern_7	42	87	0.325581	0.207921	Benign_Pattern_2	87
Infiltration_Pattern_4	14	18	0.437500	0.736842	Benign_Pattern_2	18
Infiltration_Pattern_8	243	171	0.586957	0.523707	Benign_Pattern_2	166

```

Saved:
E:\MultiStage_Attack_Detection\results\precision_recall_error_analysis.csv

PRECISION / RECALL ANALYSIS COMPLETED SUCCESSFULLY

```

Figure 5.6 Low-Precision Subpattern Error Analysis

As shown in Figure 5.6, the error analysis identified several subpatterns with precision below the predefined 0.80 threshold. The weakest results were mainly associated with Web Attacks and Infiltration subpatterns. In particular, Web_Attacks_Pattern_5 and Web_Attacks_Pattern_2 produced zero precision and recall in the evaluated samples, while several Infiltration subpatterns also showed comparatively low precision and recall values.

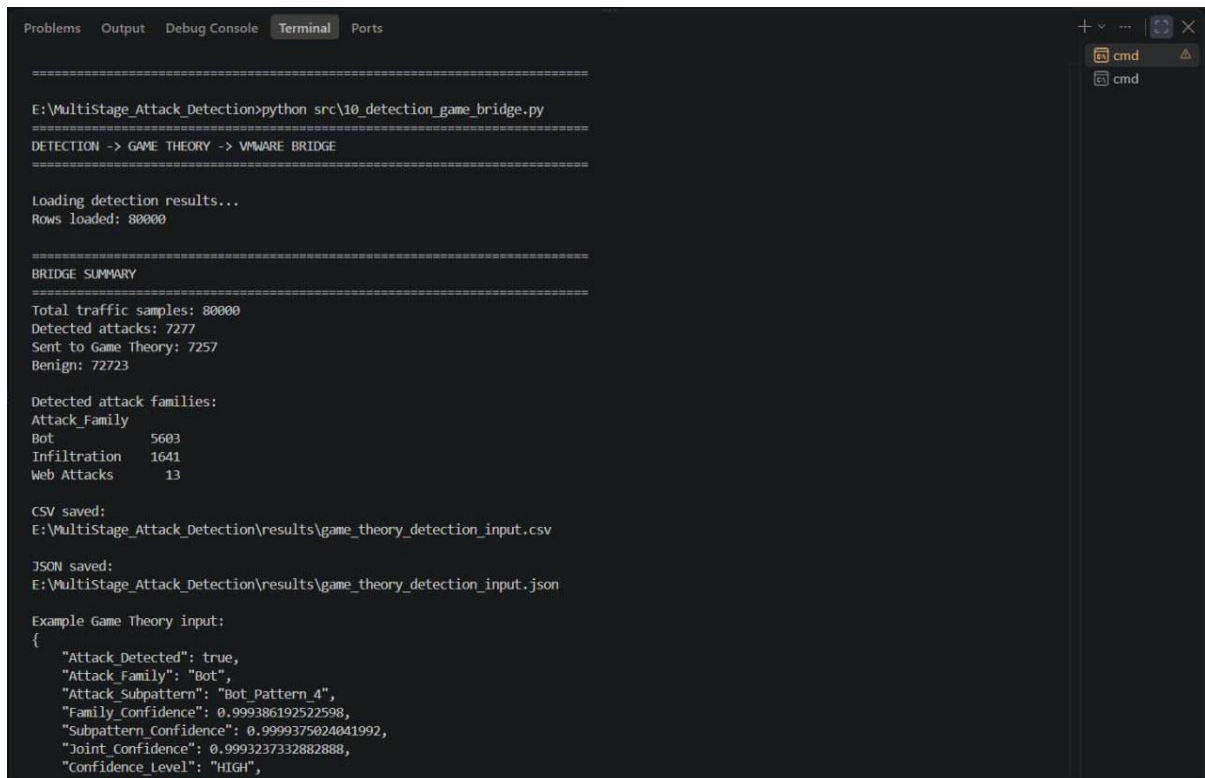
The false-positive analysis further showed that several Infiltration subpatterns were frequently confused with benign traffic. Most of these false-positive predictions originated from Benign_Pattern_1 or Benign_Pattern_2. This indicates that some Infiltration behaviors share similar feature characteristics with benign traffic, making their separation more difficult than that of the better-performing subpatterns.

Despite these difficult cases, the overall hierarchical classification performance remained high, as demonstrated by the results presented in the previous section. Therefore, the weak-subpattern analysis was used to identify the specific sources of classification error rather than to indicate a general limitation of the hierarchical model.

5.5 AI-to-VMware and Game Theory Integration Results

After completing the hierarchical classification stage, the detected attack information was transferred to the integration layer that connects the Artificial Intelligence subsystem with the VMware-based SDN environment and the Game Theory decision process. The bridge module receives the prediction output, evaluates whether the detected activity should be forwarded, and prepares the structured information required by the subsequent defense stage.

The bridge was first evaluated to verify that the hierarchical detection results were filtered according to the configured forwarding conditions and converted into the structured input expected by the Game Theory stage.



```
=====
E:\Multistage_Attack_Detection>python src\10_detection_game_bridge.py
=====
DETECTION -> GAME THEORY -> VMWARE BRIDGE
=====

Loading detection results...
Rows loaded: 80000

=====
BRIDGE SUMMARY
=====
Total traffic samples: 80000
Detected attacks: 7277
Sent to Game Theory: 7257
Benign: 72723

Detected attack families:
Attack_Family
Bot          5603
Infiltration 1641
Web Attacks  13

CSV saved:
E:\Multistage_Attack_Detection\results\game_theory_detection_input.csv

JSON saved:
E:\Multistage_Attack_Detection\results\game_theory_detection_input.json

Example Game Theory input:
{
  "Attack_Detected": true,
  "Attack_Family": "Bot",
  "Attack_Subpattern": "Bot_Pattern_4",
  "Family_Confidence": 0.999386192522598,
  "Subpattern_Confidence": 0.9999375024041992,
  "Joint_Confidence": 0.9993237332882888,
  "Confidence_Level": "HIGH",
}
```

Figure 5.7 AI Detection Bridge Summary and Game Theory Input Generation

As shown in Figure 5.7, the bridge module processed 80,000 traffic samples. A total of 7,277 samples were identified as attacks, while 72,723 samples were classified as benign traffic. Among the detected attacks, 7,257 samples satisfied the forwarding conditions and were prepared for the Game Theory stage. The detected malicious traffic was mainly associated with the Bot and Infiltration families, with a smaller number of Web Attack samples.

After the eligible AI detections were prepared, an individual accepted detection record was transmitted through the HTTP bridge to verify communication between the Windows-based AI environment and the VMware receiver.

```

Problems Output Debug Console Terminal Ports
JSON saved:
E:\MultiStage_Attack_Detection\results\game_theory_detection_input.json

Example Game Theory input:
{
  "Attack_Detected": true,
  "Attack_Family": "Bot",
  "Attack_Subpattern": "Bot_Pattern_4",
  "Family_Confidence": 0.999386192522598,
  "Subpattern_Confidence": 0.9999375024041992,
  "Joint_Confidence": 0.9993237332882888,
  "Confidence_Level": "HIGH",
  "Send_To_Game_Theory": true
}

=====
SENDING DETECTION TO VMWARE
=====
Receiver:
http://192.168.255.156:5050/detection

Sending:
{
  "attack_detected": true,
  "attack_family": "Bot",
  "attack_subpattern": "Bot_Pattern_4",
  "joint_confidence": 0.9993237332882888
}

VMware response:
{"status": "received", "saved_to": "/tmp/ml_detection_latest.json"}

Detection sent to VMware successfully.

=====
BRIDGE COMPLETED SUCCESSFULLY
=====

E:\MultiStage_Attack_Detection>

```

Figure 5.8 Successful Transmission of AI Detection Results to the VMware Environment

Figure 5.8 shows an example of the structured detection information prepared for the defense layer. The record contains the detected attack status, attack family, behavioral subpattern, family confidence, subpattern confidence, joint confidence, and confidence level. The selected record was transmitted to the VMware receiver, and the returned response confirmed that the detection information was successfully received and stored inside the virtualized SDN environment.

The receiving side was then checked independently to confirm that the transmitted detection information was not only sent by the bridge but was actually received and stored inside the Mininet VMware environment.

```

mininet@mininet-virtual-machine: ~/SDN-GameTheory-Security
mininet@mininet-virtual-machine:~/SDN-GameTheory-Security$ python3 controllers/ml_detection_receiver.py
=====
ML Detection Receiver
=====
Listening on port 5050
Endpoint: POST /detection
Waiting for Windows ML detector...
=====

===== ML DETECTION RECEIVED =====
Attack detected : True
Attack family   : Bot
Subpattern      : Bot_Pattern_4
Confidence      : 0.9993237332882888
Saved to       : /tmp/ml_detection_latest.json
=====

192.168.255.1 - - [06/Aug/2026 20:08:04] "POST /detection HTTP/1.1" 200 -

```

Figure 5.9 Successful Reception of AI Detection Results inside the Mininet VMware Environment

Figure 5.9 confirms successful reception of the transmitted detection result inside the Mininet virtual machine. The receiver obtained the attack family, behavioral subpattern, and joint confidence value and stored the received information in the local ML-detection file. The HTTP 200 response confirms that the communication between the Windows-based Artificial Intelligence subsystem and the VMware-based SDN environment was completed successfully.

Overall, the integration results demonstrate successful end-to-end transfer of Artificial Intelligence detection information from the Windows-based classification environment to the VMware-based SDN platform. The transmitted family, subpattern, and confidence information became available to the subsequent Game Theory and controller components, providing the link between detailed AI-based attack identification and adaptive network defense.

5.6 Behavioral Interpretation of Generated Subpatterns

Following the classification and error-analysis stages, the generated behavioral subpatterns were further interpreted in the original feature space. The purpose of this stage was to provide a clearer behavioral representation of the generated clusters by reconstructing their feature centers from the PCA space back to the original network-feature domain.

The interpretation module was therefore executed to verify that the cluster centers of the generated subpatterns could be reconstructed into recognizable original network features.

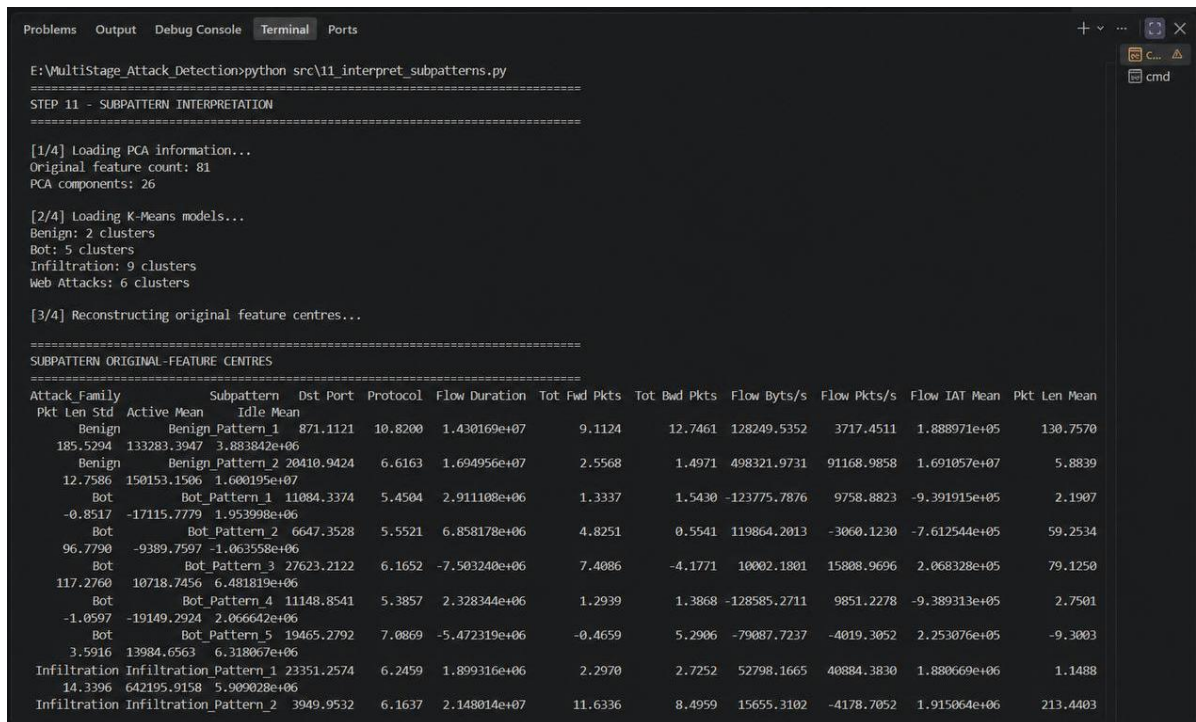


Figure 5.10 Reconstruction of Subpattern Centers in the Original Feature Space

As shown in Figure 5.10, the interpretation module loaded the PCA configuration and the family-specific K-Means models before reconstructing the cluster centers in the original feature space. The process covered 22 behavioral subpatterns distributed across the four traffic families: two Benign subpatterns, five Bot subpatterns, nine Infiltration subpatterns, and six Web Attack subpatterns.

The reconstructed centers provide representative values for several original traffic characteristics, including destination port, protocol, flow duration, forward and backward packet counts, byte and packet rates, flow inter-arrival time, packet-length statistics, active time, and idle time. These values provide a more interpretable description of the behavioral characteristics associated with each generated subpattern.

The remaining reconstructed subpatterns were then processed to confirm that the interpretation procedure covered all generated clusters rather than only a selected subset.

```

Problems Output Debug Console Terminal Ports
14.3396 642195.9158 5.909028e+06
Infiltration Infiltration_Pattern_2 3949.9532 6.1637 2.148014e+07 11.6336 8.4959 15655.3102 -4178.7052 1.915064e+06 213.4403
357.0113 181546.3276 4.974460e+06
Infiltration Infiltration_Pattern_3 -2168.2405 16.2439 -4.927219e+05 1.6311 0.5875 187045.8207 6087.7332 3.567589e+05 64.1739
32.2172 -2254.5877 1.347240e+05
Infiltration Infiltration_Pattern_4 7062.4839 5.8286 4.853452e+05 0.4415 2.8479 -170826.6009 24313.2576 9.604316e+05 -3.0815
-6.7133 4070.6731 -1.237800e+05
Infiltration Infiltration_Pattern_5 6682.8646 6.4001 2.498980e+06 8.0910 9.8037 66576.3603 -3560.8288 6.538713e+05 149.5943
297.3829 88604.9365 1.056944e+06
Infiltration Infiltration_Pattern_6 4588.5592 6.5726 1.032361e+08 27.7859 39.3060 66813.9290 -8043.5734 3.230906e+06 302.0694
466.4982 700522.8476 4.526373e+07
Infiltration Infiltration_Pattern_7 8163.2393 5.3515 2.323869e+06 1.5324 0.9483 1811132.1096 1199685.4901 -2.779978e+04 -9.5079
6.9538 -5554.1934 -1.679664e+06
Infiltration Infiltration_Pattern_8 3668.4813 6.8010 4.585344e+06 -0.1377 6.0277 9255.2178 12910.9000 -3.166994e+05 -2.4808
25.9005 333815.7440 2.279713e+06
Infiltration Infiltration_Pattern_9 15972.4389 6.2840 1.629903e+07 7.5651 3.7088 769796.8984 38445.8854 1.751115e+06 40.3800
38.9328 288012.6838 8.934322e+06
Web Attacks Web_Attacks_Pattern_1 2633.7356 4.8076 2.033747e+07 8.2333 -3.0944 82833.7632 -7746.7244 -2.325883e+06 120.2718
203.9724 -36187.4824 -8.319367e+05
Web Attacks Web_Attacks_Pattern_2 22002.5345 25.5863 3.415601e+07 -7.2551 18.1245 31185.2308 -41671.1371 2.898914e+07 507.8036
-0.1551 4172166.0295 2.219861e+07
Web Attacks Web_Attacks_Pattern_3 5632.2774 4.0819 4.915915e+07 131.2765 74.5336 290385.3864 -18137.7413 1.737001e+06 444.0026
534.0414 -64687.3775 -2.953616e+06
Web Attacks Web_Attacks_Pattern_4 6539.0552 4.3973 4.151345e+07 183.4782 111.8717 236273.4122 -18775.8101 3.161485e+06 873.0378
965.4130 -74699.0170 -1.056303e+06
Web Attacks Web_Attacks_Pattern_5 8232.5150 5.8287 1.282818e+07 1.5933 1.5362 -50097.0810 30338.9536 2.174753e+06 -1.0443
0.7822 -3230.5396 2.393086e+06
Web Attacks Web_Attacks_Pattern_6 8557.5034 5.6105 2.815003e+06 1.8895 2.0004 -92282.7356 17398.9852 -9.038487e+05 0.4469
0.1796 -14968.9250 1.736867e+06

[4/4] Saving results...

Saved:
results\subpattern_original_feature_centers.csv

Total subpatterns: 22

=====
STEP 11 COMPLETED SUCCESSFULLY
Ln 292, Col 6 Spaces: 4 UTF-8 CRLF {} Python Python 3.12 (64-bit)

```

Figure 5.11 Completed Original-Feature Interpretation of the Generated Subpatterns

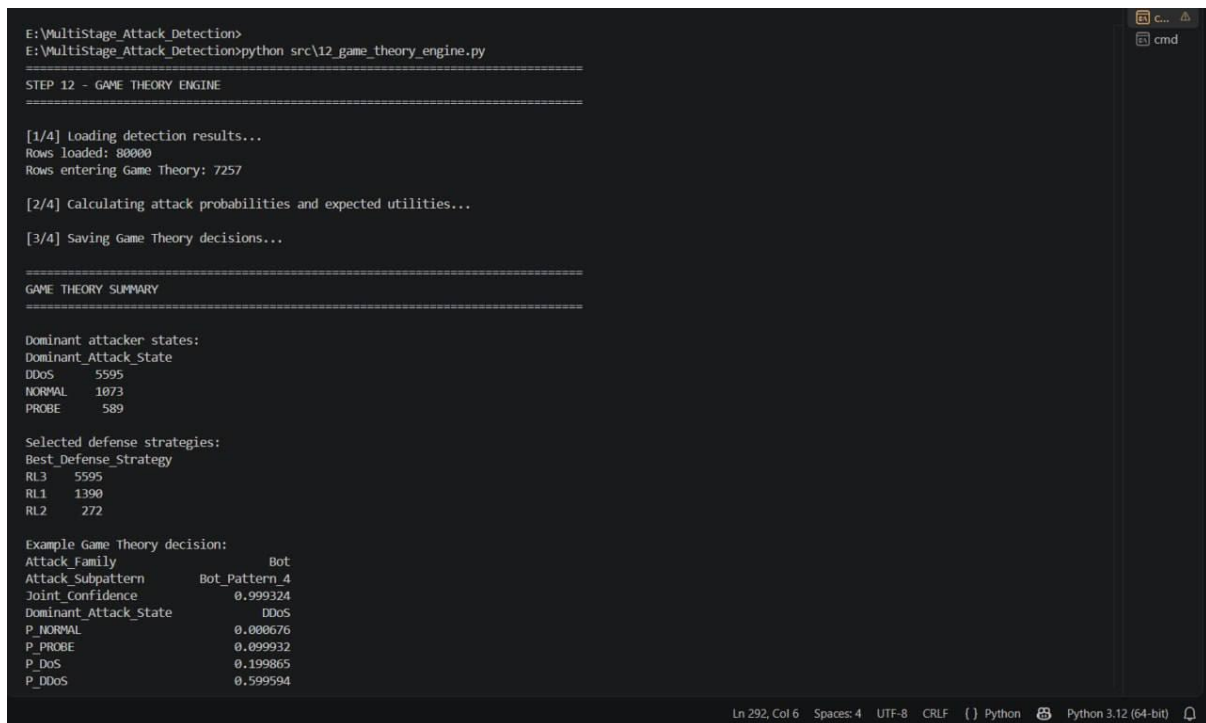
Figure 5.11 presents the remaining reconstructed subpattern centers and confirms successful completion of the interpretation stage. The resulting feature-center information was stored in the file `subpattern_original_feature_centers.csv`, and the final output confirmed that all 22 generated subpatterns were successfully processed.

This interpretation stage complements the classification results by providing a behavioral description of the generated subpatterns in terms of recognizable network features. Rather than treating the subpatterns only as cluster identifiers, the reconstructed feature centers make it possible to examine how the underlying traffic characteristics differ among the generated behavioral groups.

Overall, the subpattern-interpretation results provide an additional analytical layer for understanding the traffic behaviors identified by the Artificial Intelligence pipeline. This information supports the interpretation of the generated attack patterns before the resulting detection information is used by the subsequent Game Theory decision stage.

5.7 Game Theory Decision Results

After the AI detection results were transferred to the decision layer, the Game Theory Engine evaluated the eligible traffic samples and selected an appropriate defense strategy for each case. The decision process used the accepted AI detection information together with the available security context to estimate the probability distribution of the possible attacker states. These probabilities were then combined with the defender payoff model to calculate the expected utility of each available defense strategy and select the strategy with the highest expected utility.



```
E:\MultiStage_Attack_Detection>
E:\MultiStage_Attack_Detection>python src\12_game_theory_engine.py

=====
STEP 12 - GAME THEORY ENGINE
=====

[1/4] Loading detection results...
Rows loaded: 80000
Rows entering Game Theory: 7257

[2/4] Calculating attack probabilities and expected utilities...

[3/4] Saving Game Theory decisions...

=====
GAME THEORY SUMMARY
=====

Dominant attacker states:
Dominant_Attack_State
DDoS      5595
NORMAL    1073
PROBE      589

Selected defense strategies:
Best_Defense_Strategy
RL3      5595
RL1      1390
RL2       272

Example Game Theory decision:
Attack_Family      Bot
Attack_Subpattern  Bot_Pattern_4
Joint_Confidence   0.999324
Dominant_Attack_State DDoS
P_NORMAL           0.000676
P_PROBE            0.099932
P_DoS              0.199865
P_DDoS             0.599594
```

Figure 5.12 Game Theory Decision Summary and Selected Defense Strategies

As shown in Figure 5.12, the Game Theory Engine loaded 80,000 detection records, of which 7,257 satisfied the conditions required to enter the Game Theory decision stage. This confirms that the decision engine was applied selectively to the traffic records forwarded by the preceding AI-to-Game-Theory integration stage rather than processing all traffic samples as potential attacks.

The dominant-state analysis classified 5,595 of the evaluated records as DDoS, 1,073 as NORMAL, and 589 as PROBE. DDoS therefore represented the dominant attacker state for the majority of the records processed by the decision engine. The presence of different dominant states demonstrates that the engine did not apply a single fixed interpretation to all forwarded samples, but instead evaluated each record according to its estimated attack-state probability distribution.

The selected defense strategies also reflected the severity of the estimated attacker states. RL3 was selected for 5,595 records, RL1 for 1,390 records, and RL2 for 272 records. The predominance of RL3 indicates that the decision engine frequently selected a strong rate-limiting response for the traffic evaluated in this experiment, while less restrictive rate-limiting strategies were selected for the remaining cases.

To illustrate how an individual defense decision was produced, one forwarded Bot attack record was selected as a representative example. The purpose was to trace the decision from the AI prediction through attacker-state estimation and expected-utility calculation to the final mitigation strategy.

```

Example Game Theory decision:
Attack_Family          Bot
Attack_Subpattern      Bot_Pattern_4
Joint_Confidence       0.999324
Dominant_Attack_State  DDoS
P_NORMAL              0.000676
P_PROBE               0.099932
P_DoS                 0.199865
P_DDoS                0.599594
P_Spoofed_DDoS        0.099932
EU_ALLOW              -6.588774
EU_RL1                0.903449
EU_RL2                5.097904
EU_RL3                6.794725
EU_BLOCK              5.490887
Best_Defense_Strategy  RL3

[4/4] Completed.

CSV saved:
results\game_theory_decisions.csv

JSON saved:
results\game_theory_decisions.json

=====
STEP 12 COMPLETED SUCCESSFULLY
=====

E:\MultiStage_Attack_Detection>

```

Figure 5.13 Example Game-Theoretic Decision Based on Attack Probabilities and Expected Utilities

Figure 5.13 presents a detailed example of an individual Game Theory decision. The analyzed record was identified by the AI subsystem as Bot_Pattern_4 within the Bot attack family, with

a joint confidence of approximately 0.9993. Based on this detection information, the Game Theory Engine generated probabilities for the possible attacker states and identified DDoS as the dominant state with a probability of approximately 0.5996.

For this example, the estimated state probabilities were 0.000676 for NORMAL, 0.099932 for PROBE, 0.199865 for DoS, 0.599594 for DDoS, and 0.099932 for Spoofed_DDoS. The probability distribution therefore assigned the greatest weight to the DDoS state while retaining lower probabilities for alternative attack states.

The engine subsequently calculated the expected utility of each available defense strategy. In the presented example, the expected utilities were -6.588774 for ALLOW, 0.903449 for RL1, 5.097904 for RL2, 6.794725 for RL3, and 5.49087 for BLOCK. Since RL3 produced the highest expected utility, it was selected as the optimal defense strategy for this particular decision.

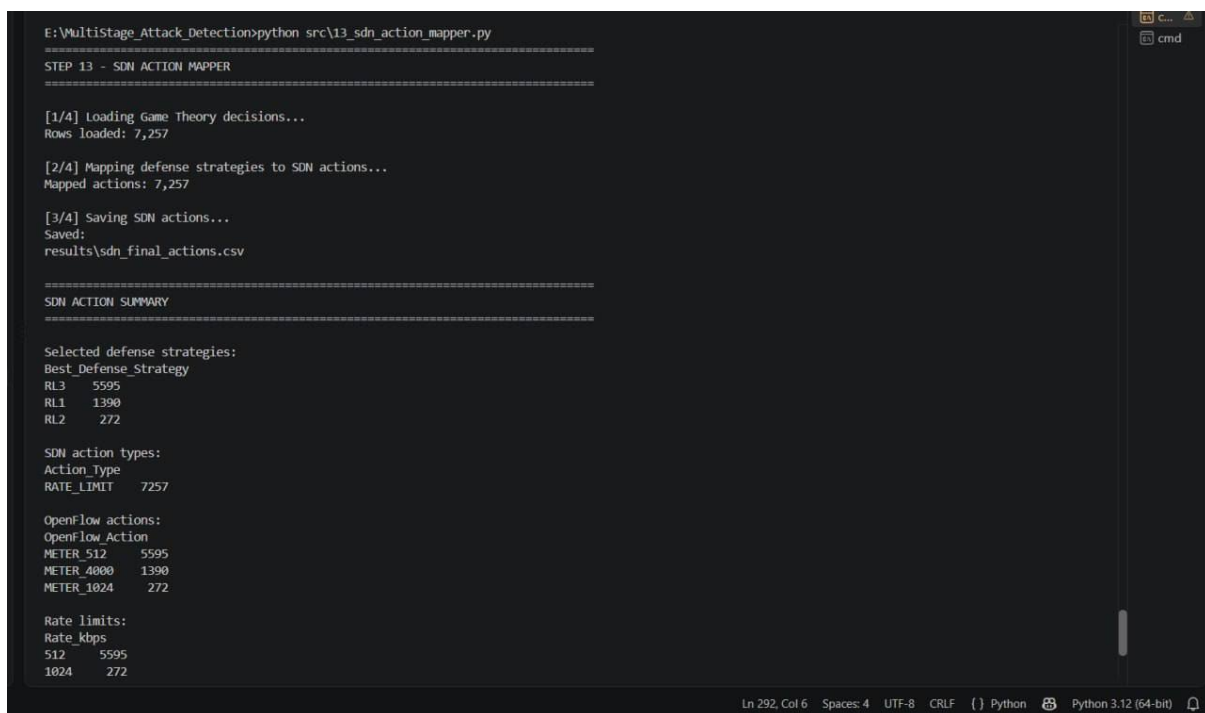
Although BLOCK also produced a positive expected utility, its value was lower than that of RL3 in this example. Consequently, the decision mechanism selected strong rate limiting rather than complete blocking. This result illustrates the adaptive nature of the proposed defense approach, in which the selected response is determined by the calculated utility rather than by automatically applying the most restrictive action.

The generated Game Theory decisions were successfully saved in both CSV and JSON formats, and the completion message shown in Figure 5.13 confirms that the full decision stage was executed successfully. These outputs provide the input required for the subsequent SDN action-mapping stage.

Overall, the results demonstrate that the Game Theory Engine successfully transformed AI-derived attack information into adaptive defense decisions. By estimating attacker-state probabilities, calculating the expected utility of multiple defensive strategies, and selecting the strategy with the highest utility, the framework provides a systematic transition from attack detection to response selection.

5.8 SDN Action Mapping Results

Following the Game Theory decision stage, the selected defense strategies were translated into executable SDN-oriented actions. The SDN Action Mapper converted each Game Theory strategy into the corresponding action type, rate limit, and OpenFlow action required for subsequent enforcement.



```
E:\MultiStage_Attack_Detection>python src\13_sdn_action_mapper.py

=====
STEP 13 - SDN ACTION MAPPER
=====

[1/4] Loading Game Theory decisions...
Rows loaded: 7,257

[2/4] Mapping defense strategies to SDN actions...
Mapped actions: 7,257

[3/4] Saving SDN actions...
Saved:
results\sdn_final_actions.csv

=====
SDN ACTION SUMMARY
=====

Selected defense strategies:
Best_Defense_Strategy
RL3      5595
RL1      1390
RL2       272

SDN action types:
Action_Type
RATE_LIMIT  7257

OpenFlow actions:
OpenFlow_Action
METER_512    5595
METER_4000   1390
METER_1024   272

Rate limits:
Rate_kbps
512      5595
1024     272
```

Figure 5.14 SDN Action Mapping Summary

As shown in Figure 5.14, all 7,257 Game Theory decisions were successfully mapped to SDN actions. The selected RL1, RL2, and RL3 strategies were translated into the corresponding OpenFlow meter actions, including METER_4000, METER_1024, and METER_512, respectively.

A single mapped record was then inspected to verify that the selected Game Theory strategy was converted into the expected SDN action and rate limit.

```
Rate limits:
Rate_kbps
512      5595
1024     272
4000     1390

Example final SDN decision:
Attack_Family      Bot
Attack_Subpattern  Bot_Pattern_4
Joint_Confidence   0.999324
Dominant_Attack_State  DDoS
Best_Defense_Strategy  RL3
Action_Type        RATE_LIMIT
Rate_kbps          512
OpenFlow_Action     METER_512

[4/4] completed.

=====
STEP 13 COMPLETED SUCCESSFULLY
=====

E:\Multistage_Attack_Detection>
```

Figure 5.15 Example Final SDN Action Generated from the Game Theory Decision

Figure 5.15 illustrates the complete mapping of an individual defense decision. For the presented Bot_Pattern_4 example, the RL3 strategy selected by the Game Theory Engine was converted into a RATE_LIMIT action with a rate of 512 kbps and the OpenFlow action METER_512. This result confirms the successful transition from strategic defense selection to an SDN-compatible mitigation action.

5.9 OpenFlow Rule Generation Results

After the SDN action-mapping stage, the resulting defense actions were converted into OpenFlow-compatible meter and flow-rule templates. This stage prepared the selected rate-limiting strategies for enforcement through Open vSwitch using OpenFlow 1.3 commands.

The final generation stage was executed to verify that the mapped rate-limiting decisions could be represented as OpenFlow 1.3 meter and flow-rule templates suitable for Open vSwitch enforcement.

```
Strategies represented:
Best_Defense_Strategy
RL3 5595
RL1 1390
RL2 272

Generated meters:
Strategy Meter_ID Rate_kbps
RL1 1 4000
RL2 2 1024
RL3 3 512

Example flow template:
Attack_Family Bot
Attack_Subpattern Bot_Pattern_1
Dominant_Attack_State DDoS
Best_Defense_Strategy RL3
Decision_Count 318
Priority 200
Meter_ID 3
OpenFlow_Action meter:3,NORMAL
Command_Template ovs-ofctl -O OpenFlow13 add-flow s1 "priority=...

[5/5] Completed.

=====
STEP 14 COMPLETED SUCCESSFULLY
=====

E:\MultiStage_Attack_Detection>
```

Figure 5.16 Generated OpenFlow Meter and Flow-Rule Template

As shown in Figure 5.16, the three rate-limiting strategies were mapped to dedicated OpenFlow meters: RL1 to 4000 kbps, RL2 to 1024 kbps, and RL3 to 512 kbps. The generated flow template also associated the selected defense strategy with the corresponding meter identifier, priority, OpenFlow action, and command template.

The successful completion of Step 14 confirms that the defense decisions produced by the previous stages were translated into OpenFlow-compatible rule templates, completing the transition from AI-based detection and Game Theory decision making to SDN-oriented mitigation preparation.

5.10 Comparison with Graduation 1

Graduation 1 established the original SDN security framework by integrating Mininet, Open vSwitch, the Ryu Controller, Snort IDS, a Snort alert-forwarding mechanism, Game Theory-based defense selection, and OpenFlow mitigation. The experimental evaluation focused mainly on ICMP flooding, spoofing-related traffic, packet loss, connectivity impact, graduated ratelimiting, and blocking enforcement.

The current phase preserves this original defense architecture and extends it with multi-stage attack tracking and Artificial Intelligence-based behavioral classification. Instead of relying

only on network observations and IDS alerts, the enhanced framework can additionally identify the general traffic family, determine its detailed behavioral subpattern, generate confidence information, and transfer this evidence to the existing Game Theory decision process.

Table 5.1 summarizes the main differences between the two project

Evaluation Aspect	Graduation 1	Current Phase	Improvement
SDN monitoring and mitigation	Snort, Ryu, Game Theory, OpenFlow	Retained and integrated with AI	Original functionality preserved
Attack representation	Mainly ICMP flooding, spoofing, probing, and severity-based states	Multi-stage temporal states plus AI family/subpattern identification	More detailed attack representation
Temporal attack tracking	Repeated decision rounds	Previous/current operational states and explicit multi-stage transition tracking	Added explicit multi-stage awareness
AI-based classification	Not implemented	Hierarchical CatBoost classification	New detection capability
Detection granularity	General network attack behavior	4 traffic families and 22 behavioral subpatterns	Subpattern-level identification added
Family classification accuracy	Not measured	98.05% on the test set	New quantitative metric
Subpattern classification accuracy	Not measured	97.28% on the test set	New quantitative metric
Macro F1-score	Not measured	93.44% on the test set	New quantitative metric
Weighted F1-score	Not measured	97.81% on the test set	New quantitative metric

Low-confidence predictions	Not measured	5.10% on the test set	Confidence-aware analysis added
AI-to-defense integration	Not available	AI family, subpattern, and confidence transferred to the defense environment	New integration layer
Game Theory input	Network and Snort observations	Network, multi-stage, and accepted AI evidence	Richer decision context
SDN enforcement	RL_1, RL_2, RL_3, BLOCK through OpenFlow	Retained with AI-supported decisions and automated action mapping	Extended decision-to-enforcement workflow

The experimental results of the current phase show that the hierarchical classifier achieved 98.05% family-level accuracy and 97.28% subpattern-level accuracy on the test set. The corresponding macro F1-score was 93.44%, while the weighted F1-score reached 97.81%. Only 5.10% of the test predictions were classified as low-confidence cases.

The integration stage processed 80,000 traffic records. Among 7,277 detected attacks, 7,257 satisfied the forwarding conditions and were prepared for the Game Theory stage, corresponding to approximately 99.73% of the detected attack records. These results demonstrate that the current phase not only adds detailed attack classification but also connects the accepted Artificial Intelligence output to the existing SDN defense process.

A direct percentage increase in classification accuracy relative to Graduation 1 cannot be calculated because the original phase did not use an Artificial Intelligence classifier and did not report family-level or subpattern-level accuracy, precision, recall, or F1-score using comparable definitions. Therefore, the improvement should be interpreted as an extension in detection capability, granularity, temporal awareness, and end-to-end integration, while the quantitative Artificial Intelligence metrics reported in the current phase serve as new evaluation measures rather than direct replacements for Graduation 1 metrics.

5.11 Chapter Summary

This chapter evaluated the enhanced framework from multi-stage attack generation and temporal state tracking through hierarchical Artificial Intelligence classification, AI-to-defense integration, Game Theory decision making, SDN action mapping, and OpenFlow rule generation.

The hierarchical CatBoost classifier achieved 98.05% family-level accuracy and 97.28% subpattern-level accuracy on the test dataset, with a macro F1-score of 93.44% and a weighted F1-score of 97.81%. Only 5.10% of the test predictions were classified as low-confidence cases. In addition, 7,257 of the 7,277 detected attack records satisfied the AI-to-defense forwarding conditions, corresponding to approximately 99.73% of the detected attacks.

Compared with Graduation 1, the current work preserves the original Snort-, Ryu-, Game Theory-, and OpenFlow-based adaptive defense framework while extending it with explicit multi-stage attack tracking, detailed family and subpattern identification, confidence-aware prediction, behavioral interpretation, and AI-to-SDN integration. Since Graduation 1 did not report comparable Artificial Intelligence classification metrics, a direct percentage increase in classification accuracy cannot be calculated; however, the current phase introduces measurable detection performance and substantially expands the level of attack detail and contextual evidence available to the defense process.

6 Chapter Six: Conclusion

This project presented the development and enhancement of an adaptive security framework for Software-Defined Networking (SDN) environments. The original phase established the practical defense architecture by integrating Mininet, Open vSwitch, the Ryu Controller, Snort IDS, a Snort alert-forwarding mechanism, Game Theory, and OpenFlow-based mitigation. This architecture provided a closed-loop workflow in which suspicious network activity could be monitored, evaluated, and followed by an adaptive response such as ALLOW, RL_1, RL_2, RL_3, or BLOCK.

The current phase extends this architecture in two major directions. First, multi-stage attack tracking was introduced to represent the progression of malicious behavior across consecutive observations rather than treating every event independently. The controller maintains previous and current operational states and uses transition information to provide additional temporal context for the defense process. Second, an Artificial Intelligence-based detection pipeline was developed using CIC-IDS-2018 traffic. The pipeline includes data preprocessing, feature engineering, PCA-based dimensionality reduction, attack-pattern segmentation, and hierarchical CatBoost classification. The final classifier identifies both the general traffic family and the corresponding behavioral subpattern while retaining confidence information for the prediction.

The experimental results demonstrated strong classification performance. On the test dataset, family-level accuracy reached 98.05%, while subpattern-level accuracy reached 97.28%. The macro F1-score was 93.44%, and the weighted F1-score reached 97.81%. Only 5.10% of the test predictions were identified as low-confidence cases. The weak-subpattern analysis also showed that classification errors were concentrated mainly in specific difficult behavioral patterns rather than representing a general weakness of the hierarchical model.

The AI detection output was successfully integrated with the existing SDN defense environment. Attack-family, subpattern, and confidence information was transferred from the Windows-based AI environment to the VMware-based Mininet environment. During the integration evaluation, 7,257 of 7,277 detected attack records, approximately 99.73%, satisfied the forwarding conditions and were prepared for the Game Theory stage. The accepted AI evidence was combined with network observations and multi-stage context, allowing the Game Theory engine to estimate attacker states, evaluate the expected utilities of the available defense strategies, and select an appropriate response. The selected strategy was

then mapped to an SDN action and converted into OpenFlow-compatible meter or flow-rule templates.

Compared with Graduation 1, the enhanced framework preserves the original Snort-, Ryu-, Game Theory-, and OpenFlow-based defense workflow while adding explicit multi-stage awareness, detailed attack-family and subpattern identification, confidence-aware prediction, and AI-to-defense integration. Graduation 1 mainly evaluated flooding, spoofing, packet loss, connectivity impact, adaptive rate limiting, and OpenFlow blocking, whereas the current phase introduces quantitative AI-based classification and finer behavioral analysis. A direct percentage increase in classification accuracy cannot be calculated because Graduation 1 did not include an AI classifier or report comparable family-level, subpattern-level, or F1-score metrics. Therefore, the main improvement is reflected in the increased detection granularity, temporal awareness, and richer decision context available to the defense mechanism.

The developed framework remains a proof-of-concept evaluated mainly in controlled and emulated environments. The SDN experiments were conducted using Mininet and VMware components, while the AI models were evaluated using prepared network-traffic datasets rather than continuous production traffic. In addition, some detailed subpatterns remained more difficult to distinguish than others.

These limitations indicate that further validation is required before applying the complete framework in a large-scale operational network.

Future work can evaluate the framework using larger SDN topologies, additional attack types, and continuous real-time traffic. Further experiments may examine controller overhead, end-to-end mitigation latency, scalability, false-positive behavior, and the impact of defensive actions on legitimate traffic. The AI component may also be improved for difficult or overlapping subpatterns, while the Game Theory payoff model can be refined using experimentally derived measurements or historical security data.

In conclusion, the current work successfully extends the adaptive SDN security framework developed in Graduation 1 by combining the existing practical defense architecture with multi-stage attack tracking and hierarchical Artificial Intelligence. The resulting framework provides an integrated transition from detailed attack identification to adaptive Game Theory decision making and programmable SDN mitigation, providing a unified experimental foundation for intelligent and context-aware network defense.

References

- [1] F. Razvan and C. Mitica, “Enhancing network security through integration of game theory in software-defined networking framework,” *International Journal of Information Security*, vol. 24, art. no. 100, 2025, doi: 10.1007/s10207-025-01012-4.
- [2] N. Gupta, S. Tanwar, and S. Badotra, “A novel sFlow based DDoS detection model in software defined networking,” *International Journal of Applied Science and Engineering*, vol. 21, no. 2, art. no. 2023510, 2024, doi: 10.6703/IJASE.202406_21(2).005.
- [3] R. Wainwright, M. Bagheri, A. Salama, and R. Saatchi, “Software-Defined Networking Security Detection Strategies and Their Limitations with a Focus on Distributed Denial-of-Service for Small to Medium-Sized Enterprises,” *Applied Sciences*, vol. 15, art. no. 12389, 2025, doi: 10.3390/app152312389.
- [4] A. Kachavimath and N. D. G., “Hybrid Approach for Detection and Mitigation of DDoS Attacks Using Multi-feature Selection, Unsupervised Learning, and Game Theory,” *Journal of Telecommunications and Information Technology*, no. 4, pp. 20–32, 2025, doi: 10.26636/jtit.2025.4.2261.
- [5] M. M. Trigui, M. S. Islam, M. D. Gambo, M. S. Alam, and T. Helmy, “Resource-Aware Ensemble Framework for DDoS Detection in SDN via Novel Game-Theoretic Feature Selection,” *IEEE Access*, vol. 14, pp. 1666–1676, 2026, doi: 10.1109/ACCESS.2025.3618519.
- [6] A. Javadpour, F. Ja’fari, T. Taleb, and C. Benzaïd, “Detecting malicious nodes using game theory and reinforcement learning in software-defined networks,” *International Journal of Information Security*, vol. 24, art. no. 117, 2025, doi: 10.1007/s10207-025-01026-y.
- [7] F. Z. Chowdhury, M. N. Adnan, M. M. Siddique, A. Parvez, and M. Y. I. B. Idris, “Advanced EDoS eye: A pragmatic model to mitigate economic denial of sustainability attack in cloud system applying dynamic game-based decision module,” *International Journal of Innovative Research and Scientific Studies*, vol. 9, no. 2, pp. 35–52, 2026, doi: 10.53894/ijirss.v9i2.11243.
- [8] S. Gong, J. Cho, and K. K. Choi, “Detection of Multi-Stage Attacks Through Attack Pattern Segmentation,” *IEEE Access*, vol. 13, pp. 204155–204167, 2025, doi: 10.1109/ACCESS.2025.3635053.

- [9] R. Kour, R. Karim, and P. Dersin, “Modelling cybersecurity strategies with game theory and cyber kill chain,” *International Journal of System Assurance Engineering and Management*, 2025, doi: 10.1007/s13198-025-02733-4.
- [10] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, “OpenFlow: Enabling Innovation in Campus Networks,” *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008, doi: 10.1145/1355734.1355746.
- [11] Open Networking Foundation, “OpenFlow Switch Specification, Version 1.3.0,” Open Networking Foundation, 2012.
- [12] D. Kreutz, F. M. V. Ramos, P. Verissimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, “Software-Defined Networking: A Comprehensive Survey,” *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015, doi: 10.1109/JPROC.2014.2371999.
- [13] B. Lantz, B. Heller, and N. McKeown, “A Network in a Laptop: Rapid Prototyping for Software-Defined Networks,” in *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks*, 2010, doi: 10.1145/1868447.1868466.
- [14] Ryu SDN Framework Community, “Ryu SDN Framework Documentation,” Ryu Project, 2026. Accessed: May 27, 2026.
- [15] Mininet Project, “Mininet Walkthrough,” Mininet Documentation, 2026. Accessed: May 27, 2026.
- [16] Open vSwitch Project, “Open vSwitch Documentation: ovs-actions,” Open vSwitch Documentation, 2026. Accessed: May 27, 2026.
- [17] Open vSwitch Project, “Open vSwitch Documentation: Configuration and Port Mirroring,” Open vSwitch Documentation, 2026. Accessed: May 27, 2026.
- [18] Cisco / Snort Team, “Snort 3 User Manual,” Snort Documentation, 2024.

- [19] Cisco / Snort Team, “Snort 3 Rule Writing Guide,” Snort Documentation, 2026. Accessed: May 27, 2026.
- [20] Scapy Project, “Scapy Documentation,” Scapy Project Documentation, 2026. Accessed: May 27, 2026.
- [21] S. Sanfilippo, “hping3(8) — hping3 Manual Page,” Debian Manpages, hping3 3.a2.ds2. Accessed: Aug. 14, 2026.
- [22] L. R. Rabiner, “A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition,” *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, 1989, doi: 10.1109/5.18626.
- [23] J. S. Bridle, “Probabilistic Interpretation of Feedforward Classification Network Outputs, with Relationships to Statistical Pattern Recognition,” in *Neurocomputing: Algorithms, Architectures and Applications*, F. Fogelman-Soulié and J. Héroult, Eds. Berlin, Germany: Springer, 1990, pp. 227–236, doi: 10.1007/978-3-642-76153-9_28.
- [24] S. W. Roberts, “Control Chart Tests Based on Geometric Moving Averages,” *Technometrics*, vol. 1, no. 3, pp. 239–250, 1959, doi: 10.1080/00401706.1959.10489860.
- [25] P. Yao, W. Hao, B. Yan, T. Yang, J. Wang, and Q. Yang, “Game-Theoretic Model for Optimal Cyber-Attack Defensive Decision-Making in Cyber-Physical Power Systems,” in *Proc. 2021 IEEE 5th Conference on Energy Internet and Energy System Integration (EI2)*, 2021, pp. 2359–2364, doi: 10.1109/EI252483.2021.9712960.